

TRƯỜNG ĐẠI HỌC GIAO THÔNG VẬN TẢI THÀNH PHỐ HỒ CHÍ MINH
VIỆN CÔNG NGHỆ THÔNG TIN VÀ ĐIỆN, ĐIỆN TỬ



BÁO CÁO THỰC HÀNH

Đề tài: Ứng dụng Offline Reinforcement Learning tối ưu thời điểm gửi giao dịch nhằm giảm chi phí gas trên Ethereum.

Môn học: Học tăng cường (Reinforcement Learning)

Mã lớp học phần: 010412710803

GVHD: TS. Nguyễn Thị Khánh Tiên

Sinh viên thực hiện:

1. Nguyễn Tiến Duy - 079205037141
2. Nguyễn Thành Long - 022205011246
3. Đỗ Nguyễn Phương Thảo - 036305007143
4. Hoàng Lê Việt Hà - 04430511246
5. Trần Thảo Dung - 079305007143
6. Võ Trần Khiêm - 052205007100

TP. HỒ CHÍ MINH, THÁNG 05/2026

Mục lục

Giới thiệu	1
Bối cảnh và Động lực	1
Phạm vi Đề tài	1
Đóng góp của Đề tài	1
1 Nội dung thực hành (Laboratory Works)	3
1.1 Giới thiệu bài Lab 1	3
1.2 Phân tích các môi trường Gymnasium	3
1.2.1 Môi trường CartPole-v1	3
1.2.2 Môi trường MountainCar-v0	4
1.2.3 Trả lời các câu hỏi phân tích	4
1.3 Đánh giá Random Agent	5
1.4 Mô hình hóa MDP và Lập trình động cho CartPole	7
1.4.1 Mô hình hóa MDP	7
1.4.2 Rời rạc hóa không gian trạng thái	7
1.4.3 Ước lượng mô hình Model-based	8
1.4.4 Triển khai thuật toán Value Iteration và Policy Iteration	9
1.4.5 Đánh giá kết quả và So sánh	10
1.5 Mục tiêu bài Lab 2	12
1.6 Thiết lập môi trường và thuật toán	12
1.6.1 Môi trường FrozenLake-v1	12
1.6.2 Cài đặt thuật toán	12
1.7 Kết quả thực nghiệm và Phân tích	13
1.8 Trả lời câu hỏi lý thuyết	14
1.8.1 Câu 1: Nguyên lý Q-learning và tính Off-policy	14
1.8.2 Câu 2: So sánh Q-learning và SARSA	14
1.8.3 Câu 3: Vai trò của chính sách ϵ -greedy	15
1.8.4 Câu 4: Thuật toán nào an toàn hơn?	15

1.8.5	Câu 5: Nguyên nhân đồ thị Reward không ổn định	15
1.9	Mục tiêu bài Lab 3	16
1.9.1	3.1 Temporal Difference Learning (TD Learning)	16
1.9.2	3.2 n-step Q-learning	18
1.9.3	3.3 Q-function Approximation với Neural Network	20
1.10	Mục tiêu bài Lab 4	22
1.11	Thiết lập và Cài đặt	22
1.11.1	4.1 Q-function Approximation với Neural Network	22
1.11.2	4.2 Deep Q-Learning và Target Network	23
1.11.3	4.3 Experience Replay	23
1.12	Kết quả thực nghiệm và Phân tích	24
2	Project: Tối ưu Phí Gas Ethereum bằng Offline Reinforcement Learning	26
2.1	Lý thuyết Nền tảng	26
2.1.1	Markov Decision Process (MDP)	26
2.1.2	Q-Learning và Phương trình Bellman	26
2.1.3	Offline Reinforcement Learning	27
2.1.4	Conservative Q-Learning (CQL)	27
2.1.5	Quy hoạch Động (Dynamic Programming)	28
2.2	Phát biểu Bài toán	28
2.2.1	Mô hình hóa Bài toán Điều phối Giao dịch	28
2.2.2	Ánh xạ Dữ liệu Chuỗi thời gian thành Bộ Transitions MDP	28
2.2.3	Không gian Trạng thái — 11 Chiều	29
2.2.4	Không gian Hành động Rời rạc	30
2.2.5	Hàm Phần thưởng	30
2.3	Triển khai Hệ thống	30
2.3.1	Hindsight Oracle — Bộ Giải Tối ưu Toàn cục	30
2.3.1.1	Động lực	30
2.3.1.2	Thuật toán DP Solver	30
2.3.1.3	Cài đặt — Numba JIT	31
2.3.2	Phân tích Dữ liệu (Data Description)	32
2.3.3	Pipeline Xây dựng Dataset	36
2.3.4	Môi trường Mô phỏng	37
2.3.5	Huấn luyện Discrete CQL	37
2.3.5.1	Cấu hình mô hình	38
2.3.5.2	Kiến trúc Q-Network	38

2.3.5.3	Chiến lược cập nhật	38
2.3.6	Đường cong Huấn luyện	38
2.4	Kết quả và Thảo luận	39
2.4.1	Phương pháp Đánh giá Chống Thiên kiến Chọn lọc (Hold-out Validation) . .	39
2.4.2	Bảng xếp hạng Hiệu năng (Leaderboard) - Tập Blind Test	40
2.4.3	Phân tích Nguyên nhân Thất bại của Thuật toán Tiêu chuẩn	40
2.4.3.1	Sự Sụp đổ của Decision Transformer (Majority Bias)	40
2.4.3.2	Bất đối xứng Rủi ro - Lợi nhuận (Risk-Reward Asymmetry)	41
2.4.4	Yếu tố Quyết định: Reward Shaping và Data Coverage	41
2.4.5	Kiến trúc Hệ thống Lai: RL + Safety Layer	41
2.4.6	Phân tích Chi tiết Từng Episode	42
2.4.7	Tính Tái lập (Reproducibility)	43
	Kết luận và Hướng phát triển	44
	Kết luận	44
	Hướng phát triển tương lai	44

Giới thiệu

Bối cảnh và Động lực

Blockchain Ethereum là một nền tảng hợp đồng thông minh phi tập trung, nơi mỗi giao dịch đều phải trả một khoản phí gọi là *gas fee* **ethereum2022eip1559, jang2026deep**. Phí này dao động liên tục theo thời gian thực, phụ thuộc vào mức độ tắc nghẽn mạng (network congestion). Trong những giờ cao điểm, chi phí gas có thể tăng gấp nhiều lần so với mức trung bình, gây ra tổn thất đáng kể cho các tổ chức và doanh nghiệp cần xử lý khối lượng giao dịch lớn định kỳ (ví dụ: các hệ thống thanh toán, sàn giao dịch phi tập trung, hoặc các ứng dụng DeFi).

Bài toán đặt ra là: *Tại mỗi thời điểm, nên quyết định gửi bao nhiêu giao dịch* (trong tổng số các giao dịch đang chờ xử lý) để đồng thời: (1) tối thiểu hóa tổng chi phí gas, và (2) không vi phạm ràng buộc về thời hạn (deadline) — tức là đảm bảo tất cả giao dịch được hoàn tất trước khi hết hạn.

Phạm vi Đề tài

Đề tài này xây dựng một **hệ thống quyết định tự động** sử dụng **Offline Reinforcement Learning (Offline RL)** để giải quyết bài toán trên. Điểm đặc thù của Offline RL là agent học hoàn toàn từ *dữ liệu lịch sử* (không tương tác trực tiếp với môi trường thật), phù hợp với ngữ cảnh blockchain nơi việc thử và sai trong môi trường thực là cực kỳ tốn kém.

Đóng góp của Đề tài

Đề tài này mang lại những đóng góp quan trọng cả về mặt lý thuyết lẫn thực tiễn trong lĩnh vực ứng dụng Trí tuệ Nhân tạo vào tối ưu hóa tài chính phi tập trung (DeFi):

- **Khung lý thuyết về Bất đối xứng Rủi ro - Lợi nhuận (Risk-Reward Asymmetry):** Đề xuất mô hình toán học giải thích hiện tượng sụp đổ (Pessimistic Collapse) của các thuật toán RL tiêu chuẩn khi đối mặt với hình phạt quá lớn, và tầm quan trọng của kỹ thuật Reward Shaping **ng1999policy**.
- **Phân tích rào cản của Conditional Behavioral Cloning:** Chứng minh sự thất bại của thuật toán tân tiến Decision Transformer (DT) **chen2021decision** trong môi trường phân bố hành động lệch (Majority Bias), so sánh với khả năng ghép quỹ đạo (Trajectory Stitching) của Q-Learning.
- **Phát triển Hệ thống Lai (Hybrid Safe-RL System):** Kết hợp thuật toán **Discrete CQL** với một **Safety Layer** (Quy tắc an toàn vật lý), đảm bảo mô hình có thể khai thác tối đa biến động giá Gas mà vẫn giữ rủi ro ở mức 0%.

- **Đạt hiệu năng Vượt trội:** Thực nghiệm trên 8 tháng dữ liệu thực tế từ Ethereum Mainnet (4.6 triệu transitions) chứng minh hệ thống tiết kiệm được **11.1% chi phí Gas** so với chiến lược Greedy truyền thống, mà vẫn đảm bảo an toàn tuyệt đối (**0% Deadline Miss Rate**).

Chapter 1

Nội dung thực hành (Laboratory Works)

1.1 Giới thiệu bài Lab 1

Mục tiêu của lab 1 như sau:

- Hiểu mô hình Agent-Environment
- Làm việc với thư viện Gymnasium
- Xây dựng Random Agent
- Hiểu cách mô hình hóa bài toán thành MDP
- Ứng dụng Dynamic Programming trên MDP rời rạc

1.2 Phân tích các môi trường Gymnasium

1.2.1 Môi trường CartPole-v1

Môi trường CartPole mô tả bài toán giữ thăng bằng cho một xe cart có gắn một cột. Mục tiêu của agent là di chuyển chiếc xe theo phương ngang để giữ cho cột luôn thẳng đứng. Các thông số của môi trường bao gồm:

- Observation space: không gian quan sát bao gồm 4 giá trị liên tục
 - **Cart Position:** Vị trí của xe cart trên mặt phẳng ngang với giới hạn $[-4.8, 4.8]$
 - **Cart Velocity:** Vận tốc của xe cart trên mặt phẳng ngang với giới hạn $[-Inf, Inf]$
 - **Pole Angle:** Góc nghiêng của cột so với phương thẳng đứng với giới hạn $[-0.418, 0.418]$
 - **Pole Angular Velocity:** Vận tốc góc của cột so với phương thẳng đứng với giới hạn $[-Inf, Inf]$
- Action space: không gian hành động bao gồm 2 giá trị rời rạc
 - **0:** Di chuyển xe cart sang trái với lực 10 Newton
 - **1:** Di chuyển xe cart sang phải với lực 10 Newton

- Reward function: hàm thưởng
 - **Thưởng +1**: cho mỗi bước thời gian mà cột luôn thẳng đứng
- Trạng thái kết thúc: Cột bị đổ hoặc xe cart ra khỏi giới hạn (trượt quá 4.8 mét so với vị trí bắt đầu), trò chơi chiến thắng khi đạt được 500 bước mà không làm đổ cột.

1.2.2 Môi trường MountainCar-v0

Môi trường MountainCar mô tả bài toán một chiếc xe bị mắc kẹt dưới đáy thung lũng và cần di chuyển để leo lên đỉnh núi. Mục tiêu của agent là di chuyển chiếc xe lên đỉnh núi. Các thông số của môi trường bao gồm:

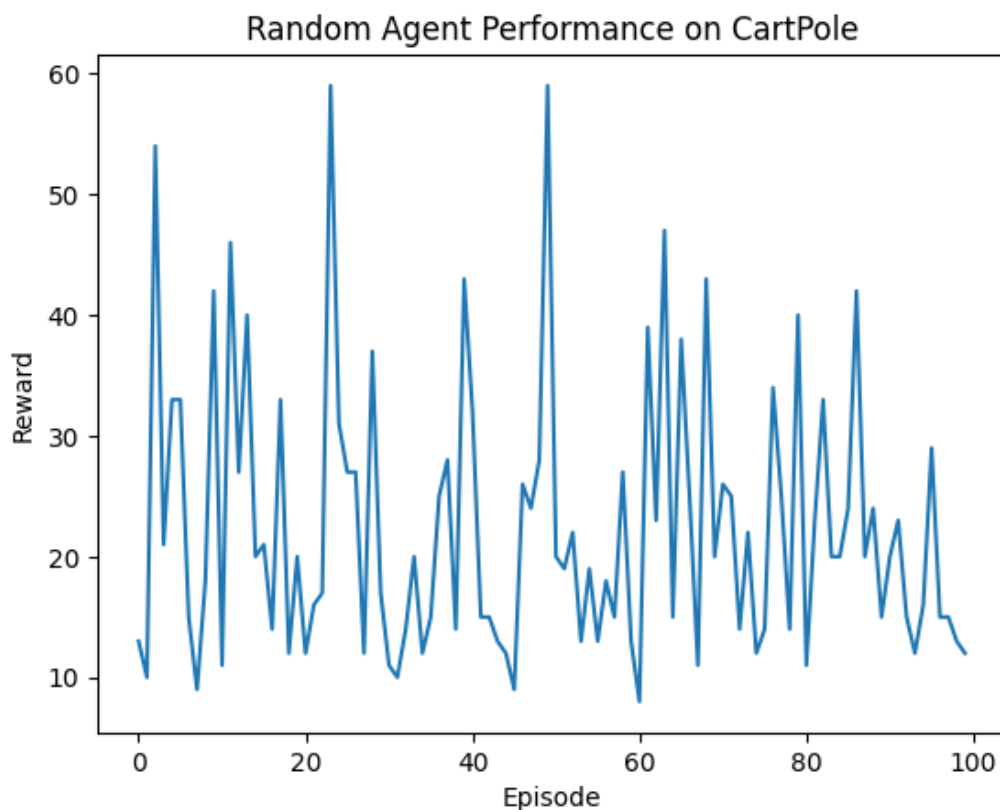
- Observation space: không gian quan sát bao gồm 2 giá trị liên tục
 - **Cart Position**: Vị trí của xe trên mặt phẳng ngang với giới hạn $[-1.2, 0.6]$
 - **Cart Velocity**: Vận tốc của xe trên mặt phẳng ngang với giới hạn $[-0.07, 0.07]$
- Action space: không gian hành động bao gồm 3 giá trị rời rạc
 - **0**: Di chuyển xe sang trái
 - **1**: Di chuyển xe sang phải
 - **2**: Không làm gì
- Reward function: hàm thưởng
 - **Thưởng -1**: cho mỗi bước thời gian
- Trạng thái kết thúc: Xe đạt đến đỉnh núi hoặc hết thời gian, trò chơi chiến thắng khi đạt được 200 bước mà không bị hỏng

1.2.3 Trả lời các câu hỏi phân tích

- **Observation có bao nhiêu chiều?** Đối với CartPole có 4 chiều, đối với MountainCar có 2 chiều
- **State space là liên tục hay rời rạc?** Cả MountainCar và CartPole đều có state space liên tục
- **Action space là liên tục hay rời rạc?** Cả MountainCar và CartPole đều có action space rời rạc
- **Reward được tính như thế nào?** Đối với MountainCar, mỗi bước đi có reward -1; còn với CartPole mỗi bước giữ gậy đứng là +1.
- **Khi nào episode kết thúc trong CartPole và MountainCar?** Với MountainCar khi xe đạt đến đỉnh núi hoặc hết thời gian (200 bước); với CartPole khi xe đạt đến giới hạn hoặc hết thời gian (500 bước).

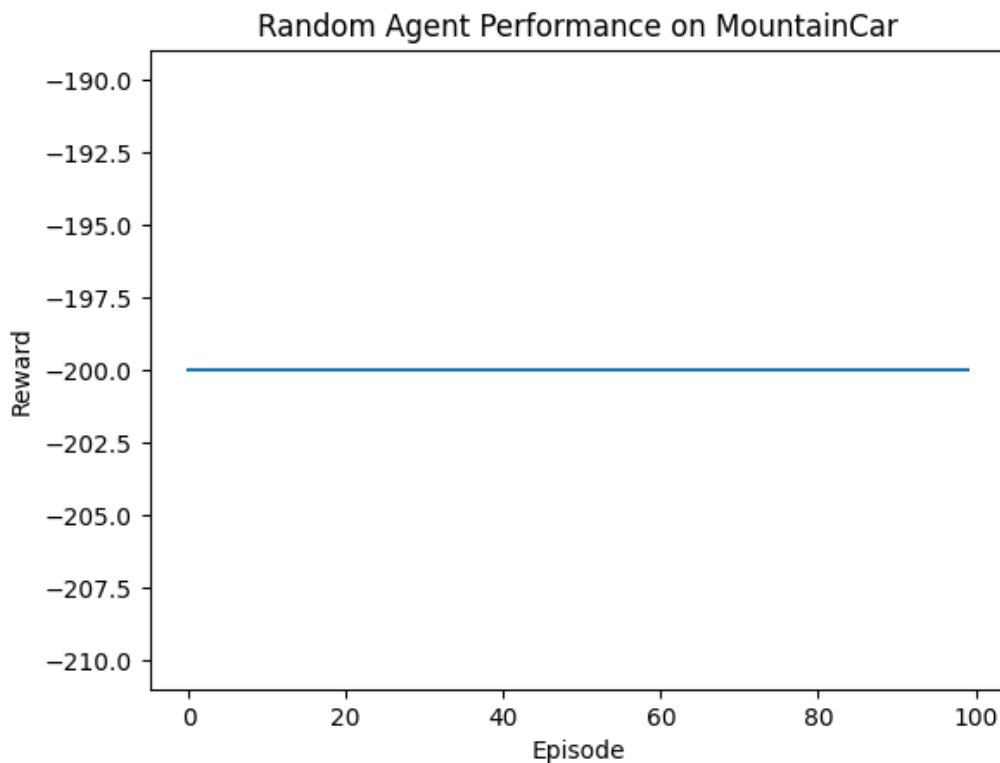
1.3 Đánh giá Random Agent

Một trong những **Policy** đơn giản nhất trong RL đó chính là chọn ngẫu nhiên hay còn gọi là **Random Policy** - tức là ta chỉ chọn hành động một cách ngẫu nhiên không quan tâm bất kỳ cái gì. Trong bài toán **CartPole**, random agent sẽ chọn hành động 0 hoặc 1 một cách ngẫu nhiên với xác suất 0.5. Khi chạy thử ta có biểu đồ reward - số bước mà agent cân bằng được cây gậy như sau:



Hình 1.1: Random Agent - CartPole

Với MountainCar, random agent sẽ chọn hành động 0, 1 hoặc 2 một cách ngẫu nhiên với xác suất 0.33. Khi chạy thử ta có biểu đồ reward - số bước mà agent leo lên dốc được như sau:



Hình 1.2: Random Agent - MountainCar

Ta thấy rằng reward luôn là -200 điều này có thể thấy agent luôn thua quá 200 bước - điều kiện để kết thúc một episode của MountainCar là xe đạt đến đỉnh núi hoặc hết thời gian (200 bước)

Phân tích kết quả:

- **Reward trung bình của random agent là bao nhiêu?** Với CartPole là ≈ 22.48 và với MountainCar là ≈ -200 .
- **Có xu hướng tăng theo thời gian không?** Khi nhìn vào cả hai biểu đồ ta có thể khẳng định là không: đối với CartPole thì dao động đáng kể và không có xu hướng tăng theo thời gian, còn với MountainCar thì hoàn toàn luôn thua - quá 200 bước, chưa bao giờ thắng 1 ván nào cả.
- **Có thể kết luận gì về agent?** Agent không học được gì từ môi trường, vì nó có policy ngẫu nhiên - mọi thứ đều diễn ra ngẫu nhiên và không có sự cải tiến.
- **Vì sao cần một policy học được?** Một policy học được là cần thiết vì nó cho phép Agent biết phân biệt giữa hành động "tốt" và "xấu" thông qua cơ chế phản hồi từ Reward. Khác với Random Policy chỉ chọn hành động mù quáng và không có tính kế thừa, một policy học được có khả năng:
 - **Tối ưu hóa hành vi:** Chuyển từ việc khám phá ngẫu nhiên sang khai thác những kinh nghiệm mang lại phần thưởng cao.
 - **Giải quyết các bài toán phức tạp:** Tìm ra những chuỗi hành động logic (như việc lấy đà trong MountainCar) mà các hành động ngẫu nhiên gần như không thể thực hiện được do xác suất quá thấp.
 - **Hội tụ:** Đảm bảo hiệu năng của Agent tăng dần theo thời gian và đạt đến trạng thái ổn định tối ưu thay vì dao động không định hướng.

1.4 Mô hình hóa MDP và Lập trình động cho CartPole

1.4.1 Mô hình hóa MDP

Môi trường CartPole được mô hình hóa dưới dạng một tiến trình quyết định Markov (MDP) với các thành phần như sau:

- **Trạng thái (S):** là một không gian liên tục 4 chiều bao gồm: vị trí xe (x), vận tốc xe ($\dot{x}$), góc nghiêng của cột (θ) và vận tốc góc của cột ($\dot{\theta}$).
- **Hành động (A):** là tập hữu hạn gồm 2 hành động rời rạc: $A = \{0, 1\}$ (Đẩy trái, đẩy phải).
- **Phần thưởng (R):** một giá trị hằng số $+1$ sau mỗi bước thời gian mà cột chưa bị đổ.
- **Điều kiện kết thúc:** Episode kết thúc khi $|\theta| > 12^\circ$ hoặc $|x| > 2.4$ mét hoặc đủ 500 bước.

Thảo luận:

- **Tính hữu hạn:** CartPole **không phải** là một MDP hữu hạn vì không gian quan sát của nó là liên tục. Việc có vô số trạng thái khiến cho tập hợp S có kích thước vô cùng lớn.
- **Khả năng áp dụng DP:** Do số lượng trạng thái là vô hạn, ta không thể áp dụng trực tiếp các thuật toán Lập trình động vốn dựa trên việc tra cứu bảng (Table-lookup). Điều này đặt ra yêu cầu phải **rời rạc hóa** không gian trạng thái ở các bước tiếp theo.

1.4.2 Rời rạc hóa không gian trạng thái

Để có thể áp dụng các thuật toán Lập trình động, ta thực hiện kỹ thuật **Binning** để chuyển đổi không gian trạng thái liên tục thành không gian rời rạc hữu hạn. **Cấu hình hệ thống:** Trong bài thực hành này, các biến trạng thái được chia theo cấu hình sau:

- Vị trí xe (Cart Position): 4 bins trong khoảng $[-4.8, 4.8]$.
- Vận tốc xe (Cart Velocity): 4 bins trong khoảng $[-5.0, 5.0]$.
- Góc nghiêng (Pole Angle): 10 bins trong khoảng $[-0.418, 0.418]$.
- Vận tốc góc (Pole Angular Velocity): 10 bins trong khoảng $[-5.0, 5.0]$.

Như vậy, tổng số trạng thái hữu hạn của mô hình là $4 \times 4 \times 10 \times 10 = 1,600$ trạng thái.

Công thức rời rạc hóa: Để chuyển đổi một giá trị liên tục s thuộc khoảng $[s_{min}, s_{max}]$ thành một chỉ số rời rạc $i \in \{0, 1, \dots, N-1\}$, ta sử dụng công thức:

$$i = \text{clip} \left(\left\lfloor \frac{s - s_{min}}{s_{max} - s_{min}} \times N \right\rfloor, 0, N - 1 \right) \quad (1.1)$$

Trong mã nguồn Python, cơ chế này được thực hiện hiệu quả bằng thư viện numpy như sau:

```

1 def discretize(state):
2     # Tính tỉ lệ vị trí của state trong khoảng [MIN, MAX]
3     ratio = (state - MIN_VALS) / (MAX_VALS - MIN_VALS)
4     # Nhân với số bins và làm tròn xuống để lấy chỉ số
5     indices = (ratio * BINS).astype(int)
6     # Giới hạn chỉ số trong khoảng [0, BINS-1]
7     indices = np.clip(indices, 0, np.array(BINS) - 1)
8     return tuple(indices)

```

Việc sử dụng `np.clip` là cực kỳ quan trọng để đảm bảo rằng nếu Agent rơi vào các trạng thái ngoại lai (outliers) vượt quá phạm vi quan sát dự kiến, chỉ số trả về vẫn nằm trong giới hạn của ma trận Q-table hoặc ma trận xác suất chuyển trạng thái.

Phân tích Trade-off: Việc lựa chọn số lượng bins là một bài toán đánh đổi quan trọng:

- **Độ chính xác (Accuracy):** Khi tăng số lượng bins, Agent có khả năng quan sát môi trường mịn hơn, giảm thiểu sai số do việc làm tròn trạng thái, từ đó có thể đạt được Policy tối ưu hơn.
- **Chi phí tính toán và Dữ liệu:** Tuy nhiên, số lượng trạng thái tăng lên sẽ khiến kích thước ma trận xác suất chuyển trạng thái $P(s'|s, a)$ tăng theo bình phương số trạng thái. Điều này không chỉ tốn tài nguyên bộ nhớ mà còn đòi hỏi số lượng mẫu thử (samples) cực lớn để ước lượng chính xác xác suất chuyển giữa các cặp trạng thái, tránh hiện tượng ma trận bị thưa (data sparsity).

1.4.3 Ước lượng mô hình Model-based

Sau khi rời rạc hóa, ta cần xây dựng mô hình toán học của môi trường thông qua việc thu thập mẫu dữ liệu trong môi trường (s, a, r, s') .

Phương pháp thu thập: Ta sử dụng một policy ngẫu nhiên (random policy) để tương tác với môi trường và thu thập dữ liệu. Trong mỗi bước thời gian, agent sẽ chọn một hành động ngẫu nhiên và ghi lại các thông tin (s, a, r, s') . **Công thức ước lượng:** Dựa trên luật số lớn, xác suất chuyển trạng thái $P(s'|s, a)$ và phần thưởng $R(s, a)$ được ước lượng như sau:

$$P(s'|s, a) = \frac{N(s, a, s')}{N(s, a)} \quad (1.2)$$

$$R(s, a) = \frac{R(s, a)}{N(s, a)} \quad (1.3)$$

Thảo luận:

- **Số lượng mẫu cần thiết:** Để mô hình có độ tin cậy cao, mỗi cặp trạng thái - hành động (s, a) cần được thăm thăm đủ số lần. Với không gian 1,600 trạng thái, việc sử dụng 60,000 episodes giúp đảm bảo hầu hết các trạng thái quan trọng gần vùng cân bằng đều được lấy mẫu đầy đủ.
- **Độ chính xác của mô hình:** Mô hình thu được chỉ là một sự xấp xỉ của môi trường thực tế. Độ chính xác phụ thuộc rất lớn vào chiến lược khám phá (exploration). Nếu Random Agent không thể chạm tới một số vùng trạng thái nhất định, ma trận P tại đó sẽ bị rỗng, dẫn đến việc Agent không thể học được hành động tối ưu tại những vùng đó.

1.4.4 Triển khai thuật toán Value Iteration và Policy Iteration

Sau khi đã có P và R , ta áp dụng thuật toán Value Iteration và Policy Iteration để tìm ra Policy tối ưu.

a. Value Iteration: Thuật toán này thực hiện vào việc tìm hàm giá trị tối ưu thông qua các bước lặp, dùng phương trình Bellman:

$$V_{k+1}(s) = \max_{a \in \mathcal{A}} \left(R(s, a) + \gamma \sum_{s' \in \mathcal{S}} P(s'|s, a) V_k(s') \right) \quad (1.4)$$

Listing 1.1: Value Iteration Implementation

```

1 def value_iteration(P, R, gamma=0.99, theta=1e-6):
2     V = np.zeros(NUM_STATES)
3     policy = np.zeros(NUM_STATES, dtype=int)
4     while True:
5         delta = 0
6         V_new = np.zeros(NUM_STATES)
7         for s in range(NUM_STATES):
8             Q_s = [R[s, a] + gamma * np.sum(P[s, a] * V) for a in range(
9                 NUM_ACTIONS)]
10            best_val = np.max(Q_s)
11            delta = max(delta, abs(best_val - V[s]))
12            V_new[s] = best_val
13            policy[s] = np.argmax(Q_s)
14        V = V_new
15        if delta < theta: break
16    return policy, V

```

Trong đó:

- $V(s)$ là hàm giá trị của trạng thái s .
- $R(s, a)$ là phần thưởng khi thực hiện hành động a trong trạng thái s .
- γ là hệ số chiết khấu.
- $P(s'|s, a)$ là xác suất chuyển từ trạng thái s sang trạng thái s' khi thực hiện hành động a .

b. Policy Iteration: Khác với Value Iteration lặp lại trên hàm giá trị, Policy Iteration chia quá trình học thành hai bước tuần tự. Điều này giúp thuật toán thường hội tụ sau rất ít vòng lặp lớn, mặc dù chi phí tính toán trong mỗi vòng lặp là cao hơn:

Bước 1: Đánh giá chính sách (Policy Evaluation) Tính toán hàm giá trị $V^\pi(s)$ cho một chính sách cố định π hiện tại mà không thực hiện việc chọn hành động mới (không có toán tử max).

$$V_{k+1}(s) = R(s, \pi(s)) + \gamma \sum_{s' \in \mathcal{S}} P(s'|s, \pi(s)) V_k(s') \quad (1.5)$$

Listing 1.2: Policy Evaluation

```

1 def policy_evaluation(policy, P, R, gamma=0.99, theta=1e-6):
2     V = np.zeros(NUM_STATES)
3     while True:
4         delta = 0
5         V_new = np.zeros(NUM_STATES)
6         for s in range(NUM_STATES):

```

```

7         a = policy[s]
8         v = R[s, a] + gamma * np.sum(P[s, a] * V)
9         delta = max(delta, abs(v - V[s]))
10        V_new[s] = v
11    V = V_new
12    if delta < theta: break
13    return V

```

Bước 2: Cải tiến chính sách (Policy Improvement) Dựa vào hàm giá trị đã được đánh giá ở Bước 1, cập nhật lại chính sách bằng cách chọn hành động đem lại kỳ vọng cao nhất cho mỗi trạng thái.

$$\pi_{new}(s) = \arg \max_{a \in \mathcal{A}} \left(R(s, a) + \gamma \sum_{s' \in \mathcal{S}} P(s'|s, a) V^{\pi}(s') \right) \quad (1.6)$$

Listing 1.3: Policy Improvement

```

1 def policy_iteration(P, R, gamma=0.99, theta=1e-6):
2     policy = np.random.choice(NUM_ACTIONS, size=NUM_STATES)
3     while True:
4         V = policy_evaluation(policy, P, R, gamma, theta)
5         policy_stable = True
6         for s in range(NUM_STATES):
7             old_action = policy[s]
8             Q_s = [R[s, a] + gamma * np.sum(P[s, a] * V) for a in range(
                NUM_ACTIONS)]
9             policy[s] = np.argmax(Q_s)
10            if old_action != policy[s]: policy_stable = False
11        if policy_stable: break
12    return policy, V

```

Quá trình lặp lại Bước 1 và Bước 2 sẽ dừng lại khi chính sách không còn thay đổi ($\pi_{new}(s) = \pi_{old}(s) \forall s \in \mathcal{S}$).

1.4.5 Đánh giá kết quả và So sánh

Sau khi tối ưu hóa các tham số rời rạc hóa (BINS = [1, 1, 24, 24]) và bổ sung bộ đếm thời gian thực thi, kết quả thực nghiệm được tổng hợp như sau:

Phương pháp	Số vòng lặp (Train)	Thời gian (s)	Reward trung bình
Random Agent	0	0.0449	22.02
Value Iteration (VI)	1,376	9.8325	42.53
Policy Iteration (PI)	20*	64.2567	12.22

Bảng 1.1: So sánh hiệu năng các thuật toán trên môi trường CartPole-v1

* Thuật toán dừng cưỡng bức ở 20 vòng lặp để tránh vòng lặp vô hạn. **Phân tích kết quả:**

- **Về tốc độ hội tụ:** Policy Iteration chạm giới hạn 20 vòng lặp mà chưa hội tụ hoàn toàn, trong khi Value Iteration cần 1,376 vòng lặp để đạt tiêu chuẩn hội tụ. Xét về thời gian thực thi thực tế, **Value Iteration nhanh hơn (9.83s)** so với Policy Iteration (64.26s). Sự chênh lệch lớn này là do mỗi vòng lặp của PI đều thực hiện bước đánh giá chính sách (Policy Evaluation) lặp đi lặp lại trên toàn bộ không gian trạng thái, dẫn đến khối lượng tính toán cực lớn.

- **Về hiệu năng điều khiển (Average Reward):**

- **Random Agent:** Đạt mức trung bình 22.02 điểm, đóng vai trò là mức cơ sở (baseline) để so sánh.
- **Value Iteration:** Đạt kết quả tốt nhất trong thí nghiệm này với **42.53 điểm** (cao gấp gần 2 lần so với Random Agent). Điều này cho thấy thuật toán đã tìm được một chính sách điều khiển có ý nghĩa từ dữ liệu mẫu.
- **Policy Iteration:** Chỉ đạt 12.22 điểm, thấp hơn cả Random Agent. Kết quả kém này là hệ quả của việc thuật toán không thể hội tụ trong 20 vòng lặp, khiến chính sách thu được vẫn còn nhiều và chưa tối ưu.

Kết luận: Thí nghiệm cho thấy mặc dù các thuật toán Lập trình động có tiềm năng tối ưu hóa chính sách, hiệu quả của chúng phụ thuộc chặt chẽ vào sự hội tụ và chất lượng mô hình MDP ước lượng. Trong trường hợp này, Value Iteration tỏ ra ổn định hơn. Sự thất bại của Policy Iteration trong việc hội tụ cũng nhấn mạnh thách thức khi áp dụng DP vào không gian trạng thái liên tục đã được rời rạc hóa, nơi các vòng lặp vô hạn hoặc sự dao động chính sách có thể xảy ra.

1.5 Mục tiêu bài Lab 2

Mục tiêu của Lab 2 nhằm giúp sinh viên:

- Hiểu cách biểu diễn bài toán Reinforcement Learning bằng Q-table.
- Nắm vững cơ chế ϵ -greedy exploration để cân bằng giữa khám phá và khai thác.
- Phân biệt giữa thuật toán On-policy (SARSA) và Off-policy (Q-learning).
- Hiểu bản chất của Temporal Difference (TD) Learning.

1.6 Thiết lập môi trường và thuật toán

1.6.1 Môi trường FrozenLake-v1

Bài toán yêu cầu Agent tìm đường từ ô START (S) đến ô GOAL (G) trên một hồ băng.

- **State space:** 16 trạng thái (4x4).
- **Action space:** 4 hành động (0: Left, 1: Down, 2: Right, 3: Up).
- **Reward:** +1 khi đến đích, 0 cho các trạng thái khác.
- **Tính chất:** Môi trường có tính ngẫu nhiên khi `is_slippery=True`, hành động của Agent có thể dẫn đến kết quả không mong muốn.

1.6.2 Cài đặt thuật toán

a. Q-Learning (Off-policy): Cập nhật giá trị Q dựa trên hành động tối ưu giả định ở trạng thái kế tiếp:

$$Q(s, a) \leftarrow Q(s, a) + \alpha[r + \gamma \max_{a'} Q(s', a') - Q(s, a)] \quad (1.7)$$

Listing 1.4: Q-Learning Update Implementation

```
1 # Q-Learning: Off-policy TD Control
2 # Q(s,a) = Q(s,a) + alpha * [R + gamma * max(Q(s',a')) - Q(s,a)]
3 q_table[state, action] = q_table[state, action] + learning_rate * \
4     (reward + gamma * np.max(q_table[new_state, :]) - q_table[state, action])
```

b. SARSA (On-policy): Cập nhật giá trị Q dựa trên hành động thực tế a' sẽ thực hiện ở trạng thái kế tiếp theo chính sách hiện tại:

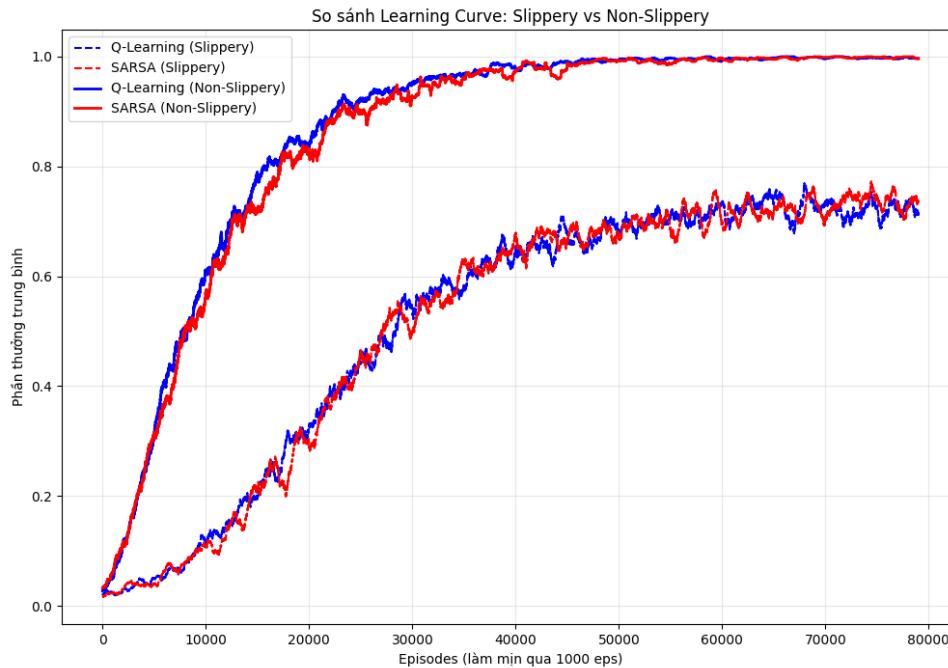
$$Q(s, a) \leftarrow Q(s, a) + \alpha[r + \gamma Q(s', a') - Q(s, a)] \quad (1.8)$$

Listing 1.5: SARSA Update Implementation

```
1 # SARSA: On-policy TD Control
2 # Rule: Q(s,a) = Q(s,a) + alpha * [R + gamma * Q(s',a') - Q(s,a)]
3 # Note: action' is chosen using the same policy as action
4 new_action = choose_action(epsilon, new_state, q_table, env)
5 q_table[state, action] = q_table[state, action] + learning_rate * \
6     (reward + gamma * q_table[new_state, new_action] - q_table[state, action])
```


1.7 Kết quả thực nghiệm và Phân tích

Trong thí nghiệm này, cả hai thuật toán được huấn luyện trong 80,000 episodes với các siêu tham số: $\alpha = 0.1$, $\gamma = 0.99$, ϵ giảm dần từ 1.0 về 0.001.



Hình 1.3: So sánh Learning Curve: Q-Learning vs SARSA (Slippy và Non-Slippy)

Phân tích đồ thị:

- **Môi trường Non-Slippy (Dễ):** Cả Q-learning và SARSA hội tụ rất nhanh về mức Reward ≈ 1.0 (tỉ lệ thắng gần 100%). Vì không có yếu tố ngẫu nhiên, Agent chỉ cần học một lần đường đi ngắn nhất là luôn thắng.
- **Môi trường Slippy (Khó) — Tại sao chỉ đạt ≈ 0.72 mà không lên 1.0?** Đây là giới hạn vật lý của môi trường, không phải lỗi của thuật toán. Khi `is_slippery=True`, mỗi hành động của Agent chỉ có xác suất $\frac{1}{3}$ đi đúng hướng mong muốn; $\frac{2}{3}$ còn lại sẽ trượt sang hai hướng vuông góc một cách ngẫu nhiên. Dù Agent đã học được Policy tối ưu tuyệt đối (biết rõ nên đi hướng nào), nó vẫn không thể kiểm soát được kết quả vật lý. Do đó, tỉ lệ thắng tối đa về mặt lý thuyết bị giới hạn bởi xác suất ngẫu nhiên của môi trường, và mức $\approx 0.72 - 0.74$ là kết quả hợp lý với cấu hình lưới 4×4 này.
- **Tại sao Reward vẫn dao động sau khi hội tụ?** Dù thuật toán đã tìm ra Policy tốt nhất, đường Reward vẫn lên xuống liên tục vì hai lý do: (1) **Epsilon còn dư:** Trong giai đoạn sau, ϵ đã giảm rất nhỏ (về 0.001) nhưng vẫn chưa bằng 0, Agent thỉnh thoảng vẫn chọn hành động ngẫu nhiên và rơi vào hố; (2) **Nhiều từ bản chất Slippy:** Ngay cả khi chọn đúng hành động, vẫn có $\frac{2}{3}$ xác suất trượt sai hướng — đây là nguồn nhiễu cố hữu không thể loại bỏ.
- **So sánh Q-learning và SARSA:** Trong môi trường Non-Slippy, cả hai cho kết quả tương đương. Trong môi trường Slippy, SARSA tuy đạt Reward cuối thấp hơn một chút so với Q-learning, nhưng lại có đường học ổn định hơn vì nó cập nhật Q-value dựa trên hành động thực

tế (bao gồm cả các bước trượt ngẫu nhiên), nên Policy của SARSA tự nhiên tránh những vị trí có rủi ro cao.

Thuật toán	Win Rate (Non-Slippery)	Win Rate (Slippery)
Q-Learning (Off-policy)	$\approx 100\%$	$\approx 74\%$
SARSA (On-policy)	$\approx 100\%$	$\approx 72\%$

Bảng 1.2: Tổng hợp Win Rate sau 80,000 episodes huấn luyện

1.8 Trả lời câu hỏi lý thuyết

1.8.1 Câu 1: Nguyên lý Q-learning và tính Off-policy

Q-learning học hàm giá trị $Q(s, a)$ bằng cách sử dụng sai số Temporal Difference. Nó được gọi là **Off-policy** vì quy tắc cập nhật của nó sử dụng giá trị của hành động tốt nhất có thể ($\max Q(s', a')$) để ước lượng phần thưởng tương lai, bất kể hành động thực tế mà Agent thực hiện ở bước tiếp theo là gì (có thể là một hành động ngẫu nhiên do ϵ -greedy).

1.8.2 Câu 2: So sánh Q-learning và SARSA

Sự khác biệt cốt lõi nằm ở **bản chất thuật toán** (chu trình thực hiện) và **thành phần mục tiêu (target)** trong công thức cập nhật:

- **Q-learning (Off-policy):**

- **Chu trình:** $s \rightarrow a \rightarrow r \rightarrow s'$. Hệ thống thực hiện cập nhật $Q(s, a)$ ngay khi quan sát được s' bằng cách lấy $\max_{a'} Q(s', a')$.
- **Bản chất:** Nó cập nhật dựa trên hành động **tối ưu giả định** ở tương lai. Hành động kế tiếp thực tế a_{t+1} chỉ được chọn sau khi đã cập nhật xong, và nó không ảnh hưởng đến giá trị target vừa tính.

- **SARSA (On-policy):**

- **Chu trình:** $s \rightarrow a \rightarrow r \rightarrow s' \rightarrow a'$. Tên gọi SARSA đại diện cho chuỗi: **S**tate, **A**ction, **R**eward, **S**tate, **A**ction.
- **Bản chất:** Hệ thống phải chọn hành động kế tiếp a' **trước** khi thực hiện cập nhật $Q(s, a)$. Giá trị target được tính dựa trên chính hành động thực tế a' đó. Do đó, nếu chính sách có yếu tố khám phá (exploration), SARSA sẽ học được cả rủi ro của những bước đi ngẫu nhiên này.

Tóm lại: Q-learning học về chính sách tối ưu (Optimal Policy), trong khi SARSA học về chính sách hiện tại (Behavior Policy), giúp nó "thận trọng" hơn trong các môi trường có hình phạt cực đoan.

1.8.3 Câu 3: Vai trò của chính sách ϵ -greedy

Chính sách này giúp cân bằng giữa **Exploration** (Khám phá môi trường mới) và **Exploitation** (Khai thác kiến thức đã học).

- **Nếu ϵ quá lớn:** Agent liên tục chọn hành động ngẫu nhiên, dẫn đến việc không bao giờ hội tụ về chính sách tối ưu.
- **Nếu ϵ quá nhỏ:** Agent ít khám phá, dễ bị kẹt vào các cực trị địa phương (local optima) và không tìm thấy đường đi tốt nhất nếu nó chưa từng đi qua.

1.8.4 Câu 4: Thuật toán nào an toàn hơn?

Trong môi trường có tính rủi ro cao, **SARSA học an toàn hơn**. Vì SARSA là on-policy, nó cập nhật Q-value dựa trên hành động thực tế (bao gồm cả các bước đi ngẫu nhiên). Nếu một đường đi tối ưu nằm ngay sát các hố nguy hiểm, SARSA sẽ nhận thấy rủi ro rơi vào hố khi khám phá và sẽ chọn một đường đi xa hơn nhưng an toàn hơn. Q-learning phớt lờ rủi ro này vì nó luôn giả định sẽ đi tối ưu ở bước sau - nhưng nó là tối ưu toán học hơn SARSA.

1.8.5 Câu 5: Nguyên nhân đồ thị Reward không ổn định

Ba nguyên nhân phổ biến và cách khắc phục:

1. **Learning rate (α) quá lớn:** Khiến giá trị Q dao động mạnh và không hội tụ. *Khắc phục:* Giảm α theo thời gian.
2. **ϵ decay quá chậm:** Agent dành quá nhiều thời gian để khám phá ngẫu nhiên ngay cả khi đã học được chính sách tốt. *Khắc phục:* Tăng tốc độ giảm ϵ .
3. **Reward quá thưa thớt (Sparse Reward):** Agent hiếm khi chạm tới đích nên không có đủ tín hiệu để cập nhật. *Khắc phục:* Tăng số lượng episodes hoặc thực hiện Reward Shaping.

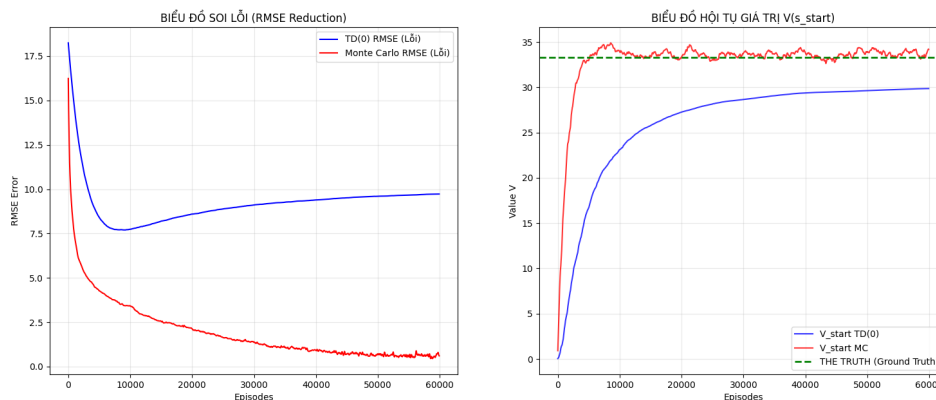
1.9 Mục tiêu bài Lab 3

Mục tiêu của Lab 3 tập trung vào các phương pháp Value-Based Reinforcement Learning nâng cao:

- Hiểu bản chất của Bootstrapping trong Temporal Difference (TD) Learning.
- Phân biệt và triển khai các biến thể: TD(0), n-step TD, và Monte Carlo.
- Ứng dụng Function Approximation với Neural Network để giải quyết bài toán không gian trạng thái liên tục.

1.9.1 3.1 Temporal Difference Learning (TD Learning)

Mô tả thực nghiệm: Chúng ta triển khai thuật toán TD(0) để học giá trị trạng thái $V(s)$ trên môi trường CartPole-v1. Vì CartPole có không gian liên tục, ta thực hiện rời rạc hóa (discretization) thành 1,600 trạng thái ($4 \times 4 \times 10 \times 10$). Kết quả được so sánh với Monte Carlo Prediction (được chạy 60,000 episodes để làm Ground Truth).



Hình 1.4: Phân tích độ ổn định và lỗi RMSE giữa TD(0) và Monte Carlo

Nhận xét đồ thị (Hình 1.4):

- **Đồ thị RMSE (trái) — Tốc độ học:** TD(0) có đường RMSE giảm nhanh và mượt mà hơn rõ rệt so với Monte Carlo trong các episode đầu tiên. Đây là hệ quả trực tiếp của Bootstrapping: TD(0) cập nhật $V(s)$ ngay sau mỗi bước đi (không cần chờ episode kết thúc), nên thông tin lan truyền nhanh hơn gấp nhiều lần so với MC vốn phải đợi đến cuối episode mới cập nhật toàn bộ.
- **Đồ thị RMSE (trái) — Mức nhiễu hội tụ:** Ở giai đoạn cuối, cả hai đều ổn định nhưng MC dao động nhiều hơn TD(0). Nguyên nhân là Variance của MC cao (tích lũy nhiễu từ toàn bộ chuỗi phần thưởng G_t), còn TD(0) chỉ phụ thuộc vào 1 bước nhiễu duy nhất ($r + \gamma V(s')$).
- **Đồ thị ổn định (phải):** Giá trị $V(s)$ của TD(0) ổn định quanh một mức nhất quán ngay từ sớm. Trong khi đó, Monte Carlo có biên độ dao động lớn hơn đáng kể vì mỗi episode lại cho một tổng thưởng G_t khác nhau tùy vào kết quả ngẫu nhiên của toàn bộ chuỗi hành động.

Trả lời câu hỏi lý thuyết:

1. **Bản chất của Bootstrapping:** TD(0) tận dụng tính đệ quy của **Phương trình Bellman** tối ưu. Việc cập nhật dựa trên giá trị của trạng thái kế tiếp được gọi là **Bootstrapping**. Về mặt lý thuyết, điều này cho phép thuật toán tận dụng cấu trúc Markov của môi trường để học hiệu quả từ dữ liệu trực tuyến (online) mà không cần hoàn tất quỹ đạo (trajectory).
2. **So sánh TD(0) và Monte Carlo (MC):** MC cập nhật dựa trên tổng phần thưởng thực tế (G_t) cuối episode, còn TD cập nhật ngay lập tức sau mỗi bước dựa trên phần thưởng tức thời và giá trị dự đoán tiếp theo ($r + \gamma V(s')$). MC là phương pháp thống kê thuần túy, trong khi TD là phương pháp lặp (iterative) để giải hệ phương trình Bellman.
3. **Phân rã Bias và Variance:** Đây là sự đánh đổi kinh điển trong học máy. MC là ước lượng **không chệch (unbiased)** vì nó sử dụng mẫu thực tế từ phân phối thực. Tuy nhiên, nó gặp vấn đề **phương sai cao (high variance)** do tích lũy nhiễu từ toàn bộ chuỗi hành động ngẫu nhiên. Ngược lại, TD(0) có **Bias cao** (phụ thuộc vào ước lượng ban đầu sai lệch) nhưng **Variance thấp** (chỉ phụ thuộc vào 1 bước nhiễu duy nhất), giúp quá trình hội tụ ổn định hơn trong các môi trường có tính ngẫu nhiên cao.
4. **Tại sao TD phù hợp online?** Vì nó cập nhật theo từng bước (step-by-step), cho phép Agent học ngay trong khi đang tương tác mà không cần chờ episode kết thúc.
5. **Ý nghĩa công thức TD(0):** $V(s) \leftarrow V(s) + \alpha[r + \gamma V(s') - V(s)]$. Trong đó α là tốc độ học, $r + \gamma V(s')$ là TD Target, và $r + \gamma V(s') - V(s)$ là TD Error.
6. **Hệ quả của Learning rate:** Nếu α quá lớn, giá trị V sẽ dao động và khó hội tụ. Nếu α quá nhỏ, tốc độ học sẽ cực chậm.
7. **Khi $\gamma \rightarrow 1$:** Agent sẽ coi trọng các phần thưởng trong tương lai xa tương đương với phần thưởng hiện tại, giá trị trạng thái sẽ lớn hơn và đại diện cho tổng thưởng dài hạn.
8. **Cài đặt $V(s)$ cho CartPole:** Sử dụng kỹ thuật rời rạc hóa (Discretization) để chia các biến liên tục thành các "thùng" (bins), biến mỗi tổ hợp thành một chỉ số trong mảng (Q-table).

9. Hàm update TD(0):

Listing 1.6: TD(0) Update Rule

```

1 # TD(0) Update:  $V(s) = V(s) + \alpha * (reward + \gamma * V(s') - V(s))$ 
2 v_next = 0 if done else v_td[next_state_idx]
3 v_td[state_idx] += alpha * (reward + gamma * v_next - v_td[state_idx])

```

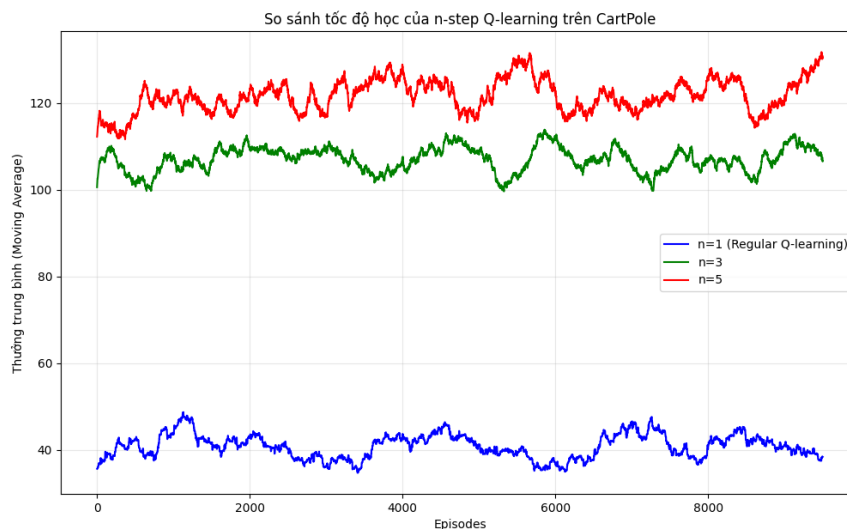
Dòng code này tính toán sai số giữa mục tiêu dự đoán ($r + \gamma V'$) và giá trị hiện tại, sau đó điều chỉnh một khoảng α để tiến gần mục tiêu hơn.

10. **Discretize trong CartPole:** Chia 4 chiều quan sát ($x, \dot{x}, \theta, \dot{\theta}$) thành các khoảng cố định bằng 'np.digitize' hoặc tính tỉ lệ theo 'MIN/MAX_VALS'.
11. **So sánh tốc độ và ổn định:** TD(0) thường hội tụ nhanh hơn về mặt số lượng bước đi và ổn định hơn MC do phương sai thấp.
12. **Thay đổi sai số:** Khi số episode tăng, RMSE giữa $V(s)$ và giá trị thực giảm dần theo dạng hàm mũ và ổn định tại một mức nhiễu thấp.
13. **Xu hướng Learning Curve:** Đồ thị RMSE (Hình 1.4 trái) cho thấy TD(0) giảm lỗi nhanh hơn MC ở giai đoạn đầu.
14. **TD(0) cho action-value?** Có thể, đó chính là thuật toán SARSA hoặc Q-learning.

15. **Vấn đề Gán trách nhiệm (Credit Assignment Problem):** Trong các môi trường có phần thưởng thưa thớt (Sparse Reward), TD(0) gặp khó khăn lớn vì tín hiệu thưởng chỉ xuất hiện ở cuối. Do TD chỉ cập nhật lùi lại đúng 1 bước mỗi lần, thông tin về phần thưởng phải ”thấm thấu” dần qua hàng ngàn episode để truyền từ trạng thái kết thúc về các trạng thái ban đầu. Điều này giải thích tại sao đồ thị học thường có giai đoạn phẳng lì trước khi đột ngột tăng vọt.

1.9.2 3.2 n-step Q-learning

Mô tả thực nghiệm: Chúng ta triển khai n-step Q-learning để trung hòa ưu nhược điểm của TD(0) và Monte Carlo. Thí nghiệm thực hiện so sánh giữa $n = 1$ (Q-learning thông thường), $n = 3$ và $n = 5$.



Hình 1.5: So sánh tốc độ học của n-step Q-learning với các giá trị n khác nhau

Nhận xét đồ thị (Hình 1.5):

- $n = 1$ (**TD thuần**): Đường Reward tăng chậm nhất. Vì mỗi bước cập nhật chỉ mang thông tin từ 1 phần thưởng tức thời r_{t+1} , tín hiệu học bị yếu và thông tin về đích phải lan truyền qua rất nhiều bước mới đến được các trạng thái ở xa — đây chính là hiện tượng Credit Assignment khó khăn.
- $n = 3$ và $n = 5$ (**n-step**): Đường Reward hội tụ nhanh hơn đáng kể và đạt mức cao hơn sớm hơn. Bằng cách tích lũy n phần thưởng thực tế trước khi Bootstrapping, mỗi bước cập nhật mang thông tin phong phú hơn (giảm Bias so với $n = 1$), giúp tín hiệu đích lan truyền nhanh hơn nhiều bước trong một lần cập nhật.
- **Tại sao không tăng n lên vô hạn?** Khi n quá lớn, thuật toán tiến dần về Monte Carlo với Variance rất cao — đường Reward trở nên dao động mạnh và mất ổn định. Giá trị n tối ưu là sự thỏa hiệp (Trade-off) giữa Bias thấp (n lớn) và Variance thấp (n nhỏ), thường nằm trong khoảng $n = 3 \sim 5$ với môi trường CartPole.

Trả lời câu hỏi lý thuyết:

1. **Bản chất toán học:** n-step Q-learning đóng vai trò là một bộ lọc điều chỉnh sự đánh đổi giữa Bias và Variance. Nó học từ n bước phần thưởng thực tế trước khi thực hiện bootstrapping, giúp giảm bớt sự lệ thuộc vào giá trị ước lượng ban đầu (giảm Approximation Bias).

2. **Sự khác biệt về Return:** TD(0) chỉ sử dụng 1 reward tức thời, MC dùng toàn bộ quỹ đạo, còn n-step sử dụng một phần thực tế (n rewards) kết hợp với giá trị dự đoán tại tương lai gần ($t+n$).
3. **Phân rã Bias/Variance khi n tăng:** Khi n tăng, Bias giảm (vì dùng nhiều dữ liệu thực hơn) nhưng Variance tăng (tích lũy nhiều nhiễu từ các hành động ngẫu nhiên trong chuỗi n bước). Giá trị n tối ưu là "điểm ngọt" giúp tối thiểu hóa tổng sai số bình phương (MSE).
4. **Công thức n-step return:** $G_{t:t+n} = r_{t+1} + \gamma r_{t+2} + \dots + \gamma^{n-1} r_{t+n} + \gamma^n \max_a Q(s_{t+n}, a)$.
5. **Vai trò max Q:** Đại diện cho giá trị kỳ vọng tốt nhất mà Agent có thể đạt được từ trạng thái thứ n trở đi.
6. **Tối ưu hóa Credit Assignment:** n-step giúp giải quyết vấn đề gán trách nhiệm hiệu quả hơn. Vì mỗi bước cập nhật tác động ngược lại đến toàn bộ n trạng thái trước đó, giúp tín hiệu phần thưởng từ đích (Goal) được truyền về nhanh hơn đáng kể so với TD(0).
7. **Cấu trúc n-step buffer:** Sử dụng một hàng đợi ('deque') lưu trữ các bộ (s, a, r) có độ dài cố định là n .
8. **Xử lý khi episode kết thúc sớm:** Nếu episode kết thúc tại bước $T < t+n$, ta tính n-step return dựa trên các phần thưởng thực tế còn lại và không thực hiện bootstrapping ($\gamma^n \max Q = 0$).
9. **Hàm tính n-step return:** Duyệt qua buffer, nhân mỗi reward với γ^i tương ứng.

Listing 1.7: n-step Return Calculation

```

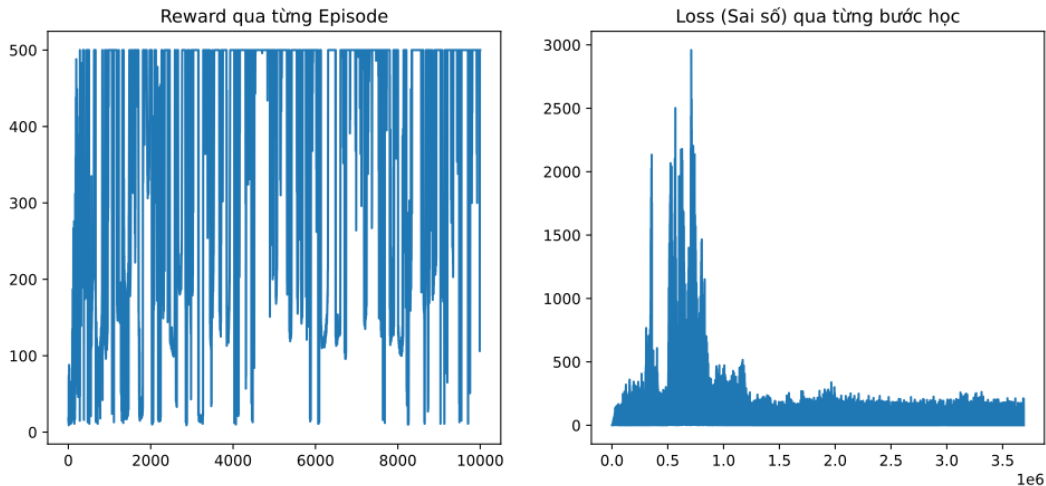
1 # Calculate n-step return G: r_t + gamma*r_{t+1} + ... + gamma^{n-1}*r_{t+n-1}
2 G = 0
3 for i, (_, _, r_i) in enumerate(buffer):
4     G += (gamma ** i) * r_i
5 # Bootstrapping term: gamma^n * max Q(s_{t+n})
6 if not done:
7     G += (gamma ** len(buffer)) * np.max(q_table[state_idx])
8 # Update rule
9 q_table[s_t, a_t] += alpha * (G - q_table[s_t, a_t])

```

10. **So sánh $n=1, 3, 5$:** Thường $n = 3$ hoặc $n = 5$ cho kết quả tốt nhất (Hình 1.5) vì tận dụng được thông tin reward dài hơn mà không bị quá nhiễu.
11. **Khi n quá lớn:** Thuật toán mất dần tính chất Bootstrapping và tiến dần về Monte Carlo, dẫn đến Variance cực cao và làm quá trình học mất ổn định do nhiễu tích lũy.
12. **Tăng tốc độ hội tụ (Convergence Rate):** n-step thường hội tụ nhanh hơn về mặt thời gian thực vì nó tận dụng được nhiều thông tin hơn trong mỗi lần cập nhật trọng số.
13. **Chiến lược chọn n :** Ta nên chọn n nhỏ cho môi trường có độ bất định cao (stochastic) để giữ variance thấp, và chọn n lớn cho môi trường ổn định hoặc có phần thưởng thưa (sparse reward) để tăng tốc độ lan truyền tín hiệu.
14. **Episode dài:** n-step rất phù hợp vì giúp đẩy nhanh tốc độ hội tụ mà không cần chờ đến cuối episode như MC.

1.9.3 3.3 Q-function Approximation với Neural Network

Mô tả thực nghiệm: Triển khai mạng Neural Network (Deep Q-Network - DQN) sử dụng PyTorch để thay thế Q-table. Chúng ta sử dụng kiến trúc 3 lớp Fully Connected với hàm kích hoạt ReLU, kết hợp Replay Buffer và kỹ thuật Double DQN để ổn định quá trình huấn luyện.



Hình 1.6: Đồ thị Reward và Loss của mô hình DQN trên CartPole-v1

Nhận xét đồ thị: Quan sát đồ thị Reward (trái) và Loss (phải), ta thấy cả hai đều có xu hướng dao động mạnh (oscillation) thay vì tăng/giảm đơn điệu và ổn định. Đây là hiện tượng đặc trưng của DQN và xuất phát từ sự tương tác của ba yếu tố chính:

- **Replay Buffer gây tương quan dữ liệu (Correlated Samples):** Replay Buffer lưu lại các bộ (s, a, r, s') từ các episode trước. Ở giai đoạn đầu, buffer chứa phần lớn các mẫu từ Policy kém (Agent chưa biết gì), khiến mạng liên tục học trên dữ liệu chất lượng thấp và Reward không thể tăng ổn định. Khi Policy được cải thiện và buffer dần được lấp đầy bằng các mẫu tốt hơn, Reward mới có xu hướng tăng dần. Tuy nhiên, nếu một mini-batch ngẫu nhiên lấy trúng nhóm mẫu xấu cũ, Reward lại tụt xuống ngay lập tức.
- **Learning Rate (α) tạo dao động quanh điểm tối ưu:** Learning Rate kiểm soát bước nhảy của Gradient Descent. Nếu α quá lớn, mỗi bước cập nhật trọng số sẽ vượt qua điểm tối ưu (overshoot), khiến Loss không giảm mà còn có thể tăng trở lại ở bước tiếp theo. Trong đồ thị Loss (phải), các đợt biến tăng vọt bất thường chính là biểu hiện của hiện tượng overshoot này. Về mặt toán học, đây chính là sự bất ổn định của Gradient Descent khi bước nhảy quá lớn so với độ cong (curvature) của hàm Loss.
- **Batch Size nhỏ làm tăng Gradient Noise:** Mỗi lần cập nhật mạng, ta chỉ lấy một mini-batch nhỏ (ví dụ 64 mẫu) thay vì toàn bộ dữ liệu. Gradient tính được từ 64 mẫu là một **ước lượng nhiễu** (noisy estimate) của Gradient thực. Điều này khiến hướng cập nhật trọng số không hoàn toàn chính xác ở mỗi bước, dẫn đến quỹ đạo học "lắc lư" xung quanh điểm hội tụ thay vì đi thẳng vào đó.

Xu hướng tổng thể: Mặc dù dao động mạnh ở từng episode, đường Reward trung bình (moving average) vẫn có xu hướng **tăng dần theo thời gian**, chứng tỏ mạng DQN vẫn đang học được Policy tốt hơn. Tương tự, đường Loss tuy có nhiều đợt biến cực bộ nhưng nhìn tổng thể vẫn có xu hướng **ổn định ở mức thấp hơn**, đặc biệt ở các episode cuối — đây là dấu hiệu của quá trình hội tụ thành công.

Trả lời câu hỏi lý thuyết:

1. **Lời nguyền đa chiều (Curse of Dimensionality):** Q-table không phù hợp vì số lượng trạng thái bùng nổ lũy thừa theo số chiều quan sát. Với CartPole (4 chiều liên tục), việc rời rạc hóa dù chỉ 10 mức mỗi chiều cũng tạo ra 10^4 trạng thái; trong các bài toán thực tế phức tạp hơn, con số này trở nên bất khả thi để lưu trữ và khám phá. Ngoài ra, Q-table thiếu khả năng **Tổng quát hóa (Generalization)** — việc học tại một trạng thái không mang lại lợi ích cho các trạng thái lân cận có tính chất tương tự.
2. **Ý tưởng Function Approximation:** Thay vì lưu giá trị cho từng ô, ta học một hàm số $Q(s, a; \theta)$ có khả năng dự đoán giá trị Q cho bất kỳ trạng thái nào dựa trên các tham số θ .
3. **Khác biệt Q-table vs Q-network:** Q-table là một mảng tra cứu tĩnh, Q-network là một mô hình toán học có khả năng học các đặc trưng và tổng quát hóa.
4. **Kiến trúc mạng:** Input là vector trạng thái (4 chiều), qua các lớp ẩn (64 nodes) dùng ReLU, Output là giá trị Q của 2 hành động.
5. **Tại sao Output node = số action?** Để trong một lần truyền xuôi (forward pass), mạng có thể tính toán đồng thời giá trị của tất cả các hành động khả thi tại trạng thái đó.
6. **Vai trò ReLU:** Thêm tính phi tuyến (non-linearity) vào mạng, giúp nó xấp xỉ được các hàm số phức tạp thay vì chỉ là các đường thẳng.
7. **Hàm loss:** Sử dụng Mean Squared Error (MSE) giữa giá trị dự đoán hiện tại $Q(s, a)$ và mục tiêu Bellman $r + \gamma \max_{a'} Q(s', a')$.
8. **Tại sao target có max Q?** Dựa trên phương trình Bellman tối ưu, khẳng định giá trị của trạng thái hiện tại bằng phần thưởng hiện tại cộng với giá trị tốt nhất có thể đạt được ở tương lai.
9. **Target thay đổi quá nhanh:** Dẫn đến hiện tượng đuổi theo mục tiêu di động (moving target), làm quá trình huấn luyện mất ổn định. Để khắc phục, ta dùng Target Network (cập nhật chậm hơn).
10. **Quan sát đồ thị Loss:** Model học tốt khi Loss giảm dần và ổn định ở một mức thấp (Hình 1.6 phải).
11. **Quan sát Reward:** Agent đạt policy tốt khi đường Reward tăng dần và tiệm cận mức tối đa của môi trường (500 điểm cho CartPole-v1).

1.10 Mục tiêu bài Lab 4

Lab 4 tập trung vào việc triển khai và tối ưu hóa thuật toán Deep Q-Network (DQN) - một cột mốc quan trọng trong Deep Reinforcement Learning:

- Hiểu cách sử dụng Neural Network để xấp xỉ hàm Q (Function Approximation).
- Triển khai cơ chế Experience Replay để xóa bỏ sự tương quan giữa các mẫu dữ liệu.
- Sử dụng Target Network để ổn định quá trình tối ưu hóa mục tiêu Bellman.
- Nắm vững các kỹ thuật ổn định training như Soft Update và Double DQN.

Listing 1.8: Soft Update & DQN Learn

```

1 def learn(self, experiences, gamma):
2     states, actions, rewards, next_states, dones = experiences
3     # Double DQN action selection & evaluation
4     best_action = self.qnetwork_local(next_states).argmax(1).unsqueeze(1)
5     Q_targets_next = self.qnetwork_target(next_states).gather(1, best_action)
6     # Compute targets and expected values
7     Q_targets = rewards + (gamma * Q_targets_next * (1 - dones))
8     Q_expected = self.qnetwork_local(states).gather(1, actions)
9     # Loss & Optimization
10    loss = F.mse_loss(Q_expected, Q_targets)
11    self.optimizer.zero_grad(); loss.backward(); self.optimizer.step()
12    # Update target network
13    self.soft_update(self.qnetwork_local, self.qnetwork_target, TAU)
14
15 def soft_update(self, local_model, target_model, tau):
16     # theta_target = tau * theta_local + (1 - tau) * theta_target
17     for target_param, local_param in zip(target_model.parameters(), local_model.
18         parameters()):
19         target_param.data.copy_(tau * local_param.data + (1.0 - tau) *
20             target_param.data)

```

1.11 Thiết lập và Cài đặt

1.11.1 4.1 Q-function Approximation với Neural Network

Thay vì lưu trữ Q-table, chúng ta sử dụng một mạng Neural (MLP) với kiến trúc:

- **Input:** Vector trạng thái 4 chiều (CartPole-v1).
- **Hidden Layers:** 2 lớp ẩn, mỗi lớp 128 units, sử dụng hàm kích hoạt ReLU.
- **Output:** 2 nodes tương ứng với giá trị Q của các hành động (Trái/Phải).

Trả lời câu hỏi lý thuyết:

1. **Tại sao Q-table không dùng được cho state liên tục?** Vì số lượng trạng thái là vô hạn, không thể biểu diễn bằng một bảng hữu hạn. Ngoài ra, Q-table không thể "tổng quát hóa" kiến thức từ trạng thái này sang trạng thái lân cận.

2. **Function Approximation là gì?** Là việc sử dụng một hàm số có tham số (như mạng Neural) để xấp xỉ giá trị của một hàm số phức tạp (hàm Q). Thay vì lưu từng giá trị đơn lẻ, ta chỉ cần lưu bộ trọng số θ .
3. **Tại sao Output layer = số action?** Để trong một lần forward pass, mạng có thể dự đoán đồng thời giá trị Q cho tất cả các hành động khả thi, giúp việc chọn hành động (argmax) hiệu quả hơn.
4. **Vai trò của ReLU?** ReLU giúp mạng học được các mối quan hệ phi tuyến tính phức tạp giữa trạng thái và giá trị Q, điều mà các hàm tuyến tính không làm được.
5. **Neural Network giúp Generalize như thế nào?** Các trạng thái gần nhau trong không gian vector sẽ kích hoạt các neuron tương tự nhau, dẫn đến đầu ra Q-value tương tự. Do đó, Agent có thể đưa ra quyết định hợp lý cho cả những trạng thái nó chưa từng gặp trực tiếp.

1.11.2 4.2 Deep Q-Learning và Target Network

Thuật toán DQN cập nhật trọng số dựa trên cực tiểu hóa hàm Loss:

$$L = \mathbb{E} \left[(r + \gamma \max_{a'} Q_{target}(s', a') - Q_{local}(s, a))^2 \right] \quad (1.9)$$

Trả lời câu hỏi lý thuyết:

1. **Tại sao target dùng max Q(s', a)?** Dựa trên phương trình Bellman tối ưu, khẳng định rằng giá trị của trạng thái hiện tại bằng phần thưởng tức thời cộng với giá trị kỳ vọng tốt nhất từ trạng thái kế tiếp.
2. **Bootstrapping trong DQN là gì?** Là việc sử dụng dự đoán của chính mạng Neural ở tương lai ($Q(s', a')$) để cập nhật giá trị hiện tại.
3. **Nếu target thay đổi liên tục?** Quá trình học sẽ cực kỳ mất ổn định (hiện tượng "đuổi theo mục tiêu di động"). Target Network giúp cố định mục tiêu này trong một khoảng thời gian để mạng Local có thể hội tụ.
4. **So sánh với Q-learning (tabular):** DQN có khả năng xử lý không gian trạng thái khổng lồ và khả năng tổng quát hóa, trong khi Tabular Q-learning chỉ hoạt động tốt trên các môi trường rất nhỏ và rời rạc.

1.11.3 4.3 Experience Replay

Cơ chế Experience Replay lưu trữ bộ $(s, a, r, s', done)$ vào một Buffer và lấy mẫu ngẫu nhiên (Mini-batch) để huấn luyện.

Listing 1.9: Experience Replay Implementation

```

1 class ReplayMemory:
2     def __init__(self, action_size, buffer_size, batch_size, device):
3         self.memory = deque(maxlen=buffer_size)
4         self.batch_size = batch_size
5     def add(self, state, action, reward, next_state, done):
6         self.memory.append((state, action, reward, next_state, done))
7     def sample(self):

```

```

8      # Shuffling to break temporal correlation
9      experiences = random.sample(self.memory, k=self.batch_size)
10     # Convert to Tensors...
11     return (states, actions, rewards, next_states, dones)

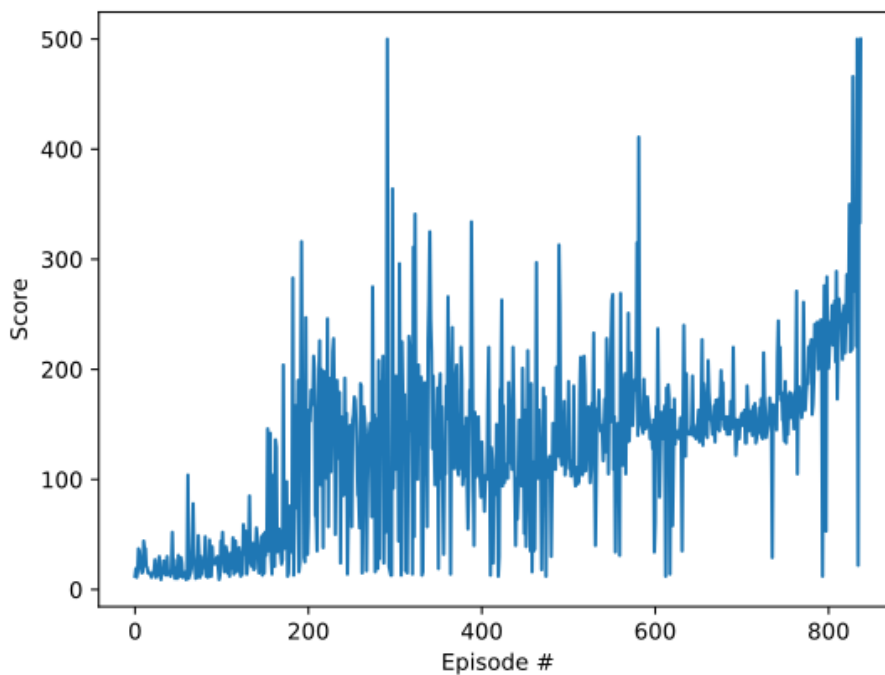
```

Trả lời câu hỏi lý thuyết:

1. **Vi phạm giả định i.i.d:** Các thuật toán tối ưu hóa dựa trên Gradient (như Adam, SGD) đều dựa trên giả định ngầm rằng dữ liệu huấn luyện là **độc lập và phân phối chuẩn (i.i.d)**. Trong RL, các trải nghiệm kế tiếp trong một episode có sự tương quan thời gian (temporal correlation) cực cao (s_{t+1} phụ thuộc vào s_t).
2. **Hậu quả của sự tương quan:** Nếu huấn luyện trực tiếp theo chuỗi, Gradient của mỗi mini-batch sẽ bị chệch (biased) theo quỹ đạo hiện tại, dẫn đến việc mạng Neural bị "học vẹt" con đường vừa đi qua và xảy ra hiện tượng **Quên thảm họa (Catastrophic Forgetting)** đối với các kinh nghiệm cũ.
3. **Cơ chế phá vỡ tương quan của Replay Buffer:** Bằng cách lưu trữ và lấy mẫu ngẫu nhiên (shuffling), Replay Buffer biến dữ liệu chuỗi thời gian thành một tập hợp "có vẻ" độc lập, giúp ước lượng Gradient trở nên không chệch và ổn định quá trình hội tụ.
4. **Đánh đổi về kích thước Buffer:** Buffer quá nhỏ sẽ không đủ sức phá vỡ tương quan. Buffer quá lớn có thể chứa quá nhiều dữ liệu "lỗi thời" từ các chính sách cũ (Stale data), làm chậm tốc độ cải thiện của chính sách hiện tại.

1.12 Kết quả thực nghiệm và Phân tích

Trong bài Lab này, chúng ta đã triển khai **Double DQN** kết hợp với **Soft Update** ($\tau = 0.001$). Kết quả cho thấy Agent đạt mức điểm tối đa 500/500 chỉ sau khoảng 400-600 episodes.



Hình 1.7: Learning Curve của DQN trên môi trường CartPole-v1

Nhận xét đồ thị (Hình 1.7):

- **Tốc độ hội tụ:** Agent đạt mức Reward tối đa 500/500 điểm chỉ sau khoảng 400-600 episodes. Đây là kết quả vượt trội so với DQN thông thường ở Lab 3, nơi Reward dao động liên tục và không ổn định xuyên suốt quá trình huấn luyện.
- **Tại sao ổn định hơn Lab 3? — Vai trò của Double DQN:** DQN thông thường dùng cùng một mạng để vừa *chọn hành động* vừa *định giá trị*, dẫn đến Overestimation Bias (đánh giá quá cao các Q-value nhiều). Double DQN tách hai nhiệm vụ này: mạng **Local** chọn hành động ($\arg \max$), còn mạng **Target** định giá trị cho hành động đó. Sự phân tách này triệt tiêu phần lớn Overestimation Bias, khiến các đột biến Loss và Reward giảm đi đáng kể.
- **Tại sao ổn định hơn Lab 3? — Vai trò của Soft Update ($\tau = 0.001$):** Thay vì sao chép nguyên xi trọng số từ mạng Local sang mạng Target sau mỗi C bước (Hard Update), Soft Update chỉ dịch chuyển mạng Target một lượng rất nhỏ ($\theta_{target} \leftarrow \tau \theta_{local} + (1 - \tau) \theta_{target}$). Điều này loại bỏ hoàn toàn hiện tượng mục tiêu nhảy vọt đột ngột (moving target problem), khiến Loss giảm đều đặn hơn thay vì bị các đột biến bất thường như Lab 3.
- **Giai đoạn ổn định (Plateau):** Sau khi đạt 500 điểm, đường Reward duy trì ổn định ở mức tối đa mà không bị tụt xuống — đây là bằng chứng cho thấy Policy đã hội tụ thực sự chứ không chỉ là may mắn ngẫu nhiên.

Trả lời câu hỏi phân tích:

1. **Khi nào model học tốt (loss)?** Khi hàm Loss giảm dần và dao động quanh một giá trị thấp, đồng thời xu hướng chung của Reward là tăng lên. Lưu ý: Loss trong RL không phản ánh trực tiếp chất lượng Policy như trong Supervised Learning.

2. **Khi nào agent đạt policy tốt (reward)?** Khi đường Reward trung bình tiệm cận mức trần của môi trường (500 điểm) và duy trì ổn định qua nhiều episodes.
3. **Sự bất ổn định và "The Deadly Triad":** Deep RL khó hội tụ hơn Tabular RL do sự kết hợp của 3 yếu tố (Tam giác chết): (1) **Function Approximation** (thay đổi một trọng số ảnh hưởng toàn bộ không gian trạng thái); (2) **Bootstrapping** (dùng dự đoán để học dự đoán); và (3) **Off-policy training** (dữ liệu học khác với chính sách hiện tại). Sự kết hợp này dễ dẫn đến việc Q-value bị khuếch đại sai số và phân kỳ.
4. **Bản chất toán học của Overestimation Bias:** Toán tử max trong mục tiêu Bellman luôn chọn giá trị lớn nhất, điều này có xu hướng chọn cả những sai số dương (positive noise) trong ước lượng Q-value. Hệ quả là mạng DQN thường đánh giá quá cao giá trị của các hành động. Chúng ta đã khắc phục điều này bằng **Double DQN**: dùng mạng Online để chọn hành động (argmax) and mạng Target để định giá, giúp tách biệt việc chọn lọc và định giá, từ đó triệt tiêu sai số hệ thống này.

Chapter 2

Project: Tối ưu Phí Gas Ethereum bằng Offline Reinforcement Learning

2.1 Lý thuyết Nền tảng

2.1.1 Markov Decision Process (MDP)

Bài toán điều phối giao dịch được mô hình hóa dưới dạng **Markov Decision Process** (MDP), được định nghĩa bởi bộ $(\mathcal{S}, \mathcal{A}, P, R, \gamma)$:

- $\mathcal{S}$: Không gian trạng thái — mô tả trạng thái hiện tại của hệ thống.
- $\mathcal{A}$: Không gian hành động — tập các quyết định có thể thực hiện.
- $P(s'|s, a)$: Hàm chuyển trạng thái — xác suất chuyển sang s' khi ở s thực hiện a .
- $R(s, a)$: Hàm phần thưởng — tín hiệu phản hồi từ môi trường.
- $\gamma \in [0, 1)$: Hệ số chiết khấu tương lai.

Mục tiêu là tìm chính sách (policy) $\pi : \mathcal{S} \rightarrow \mathcal{A}$ tối đa hóa tổng phần thưởng kỳ vọng chiết khấu:

$$J(\pi) = \mathbb{E}_{\pi} \left[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t) \right] \quad (2.1)$$

Đây chính là nền tảng để xây dựng bài toán RL nơi mà Dữ liệu/Môi trường phải đưa về dạng có $\mathcal{S}$ (state) và $\mathcal{A}$ (action).

2.1.2 Q-Learning và Phương trình Bellman

Q-function (hay Action-Value function) $Q^{\pi}(s, a)$ biểu diễn tổng phần thưởng kỳ vọng khi ở trạng thái s , thực hiện hành động a , và sau đó tuân theo chính sách π :

$$Q^{\pi}(s, a) = \mathbb{E}_{\pi} \left[\sum_{k=0}^{\infty} \gamma^k R(s_{t+k}, a_{t+k}) \mid s_t = s, a_t = a \right] \quad (2.2)$$

Phương trình Bellman biểu diễn tính nhất quán đệ quy của Q -function:

$$Q^\pi(s, a) = R(s, a) + \gamma \sum_{s'} P(s'|s, a) Q^\pi(s', \pi(s')) \quad (2.3)$$

Q -function tối ưu $Q^*(s, a) = \max_\pi Q^\pi(s, a)$ thỏa:

$$Q^*(s, a) = R(s, a) + \gamma \sum_{s'} P(s'|s, a) \max_{a'} Q^*(s', a') \quad (2.4)$$

Chính sách tối ưu: $\pi^*(s) = \arg \max_a Q^*(s, a)$.

2.1.3 Offline Reinforcement Learning

Trong **Online RL**, agent học bằng cách tương tác trực tiếp với môi trường — mỗi hành động tạo ra dữ liệu mới. Trong **Offline RL** [levine2020offline](#), agent chỉ được học từ tập dữ liệu lịch sử cố định:

$$\mathcal{D} = \{(s_i, a_i, r_i, s'_i)\}_{i=1}^N \quad (2.5)$$

thu thập bởi một chính sách hành vi (behavior policy) π_β — không nhất thiết là tối ưu.

Thách thức cốt lõi: *Distribution shift* hay *Extrapolation Error*. Khi agent muốn đánh giá Q -value của một hành động $a_{\text{OOD}} \notin \mathcal{D}$, mạng Q_θ được huấn luyện bằng dữ liệu từ π_β sẽ *extrapolate* không chính xác và thường overestimate $Q_\theta(s, a_{\text{OOD}})$. Điều này dẫn đến policy chọn các hành động ”ảo tưởng” không có trong data, gây sụp đổ.

2.1.4 Conservative Q-Learning (CQL)

Discrete CQL [kumar2020conservative](#) giải quyết extrapolation error bằng cách thêm một *conservative penalty* trực tiếp vào hàm mục tiêu huấn luyện:

$$\mathcal{L}_{\text{CQL}}(\theta) = \underbrace{\mathbb{E}_{(s,a,r,s') \sim \mathcal{D}} [(Q_\theta(s, a) - y)^2]}_{\mathcal{L}_{\text{TD}} - \text{Bellman Error}} + \alpha \underbrace{\mathbb{E}_{s \sim \mathcal{D}} \left[\log \sum_{a'} e^{Q_\theta(s, a')} - \mathbb{E}_{a \sim \pi_\beta} [Q_\theta(s, a)] \right]}_{\text{Conservative Penalty}} \quad (2.6)$$

Trong đó $y = r + \gamma \max_{a'} Q_{\bar{\theta}}(s', a')$ là Bellman target (dùng target network $\bar{\theta}$).

Ý nghĩa hình học của Conservative Penalty:

- Số hạng $\log \sum_{a'} e^{Q_\theta(s, a')}$: *softmax* của tất cả Q -values $\rightarrow$ tác động như lực kéo xuống đều lên mọi hành động.
- Số hạng $\mathbb{E}_{a \sim \pi_\beta} [Q_\theta(s, a)]$: lực đẩy lên đối với các hành động có trong data.
- Hiệu số: Q -values của OOD actions bị hạ thấp, Q -values của in-distribution actions được nâng cao.

Kết quả: policy học được sẽ ưa thích các hành động đã được quan sát trong $\mathcal{D}$ — đặc biệt là những hành động từ Oracle có reward cao.

2.1.5 Quy hoạch Động (Dynamic Programming)

Backward Induction là phương pháp DP tìm chính sách tối ưu *toàn cục* trong bài toán hữu hạn bước T . Thuật toán bắt đầu từ bước cuối và tính ngược về bước đầu:

Khởi tạo: $V[T, Q] =$ chi phí tối thiểu khi chỉ còn Q giao dịch và đã hết thời gian.

Bước quy hoạch (với $t = T - 1, T - 2, \dots, 0$):

$$V[t, Q] = \min_{b \in \mathcal{A}} \{c(t, Q, b) + V[t + 1, f(Q, b, t)]\} \quad (2.7)$$

Trong đó:

- $c(t, Q, b)$: chi phí tức thời khi ở (t, Q) chọn bin b — gồm gas cost và urgency penalty.
- $f(Q, b, t) = \max(0, Q - n_b) + w_{t+1}$: hàm chuyển trạng thái queue.

DP đảm bảo kết quả là **optimal global strategy** vì nó xét *toàn bộ* quỹ đạo tương lai có thể xảy ra.

2.2 Phát biểu Bài toán

2.2.1 Mô hình hóa Bài toán Điều phối Giao dịch

Xét một tổ chức cần xử lý Q_0 giao dịch trên Ethereum trong khoảng thời gian $[0, T]$ ($T = 24$ giờ). Tại mỗi block (bước thời gian t), hệ thống phải quyết định số lượng giao dịch cần gửi.

MDP của bài toán:

- **State:** $s_t = (g_t, g_{t-1}, g_{t-2}, p_t, m_t, a_t, u_t, b_t, Q_t, \tau_t, \bar{g}_t) \in \mathbb{R}^{11}$
- **Action:** $a_t \in \{0, 1, 2, 3, 4\}$ — bin ID tương ứng tỷ lệ xả $\{0\%, 25\%, 50\%, 75\%, 100\%\}$
- **Transition:** $Q_{t+1} = \max(0, Q_t - n_t) + w_{t+1}$
- **Reward:** r_t được thiết kế để tối thiểu hóa chi phí gas và phạt deadline
- **Horizon:** $T = 24$ giờ (1 episode = 1 ngày)

Bài toán tối ưu:

$$\min_{\pi} \mathbb{E} \left[\sum_{t=0}^{T-1} \frac{g_t}{\sigma} (C_{\text{base}} \cdot \mathbf{1}[n_t > 0] + C_{\text{mar}} \cdot n_t) + \frac{\lambda}{\sigma} \cdot \mathbf{1}[Q_T > 0] \right] \quad (2.8)$$

2.2.2 Ánh xạ Dữ liệu Chuỗi thời gian thành Bộ Transitions MDP

Trong thực tế, dữ liệu thu thập từ blockchain Ethereum là dữ liệu chuỗi thời gian (time-series) thô. Để áp dụng Offline RL, ta phải biến đổi các log dữ liệu này thành tập các bộ chuyển trạng thái (Transitions) $\mathcal{D} = \{(s_t, a_t, r_t, s_{t+1})\}$. Quy trình ánh xạ từ dữ liệu thô sang các thành phần của MDP được thực hiện như sau:

1. **Khởi tạo Trạng thái** ($s_t \in \mathcal{S}$): Tại mỗi block t , từ các dữ liệu thô như giá gas và số lượng giao dịch, ta áp dụng Feature Engineering để tính toán ra vector 11 chiều (đã mô tả ở Bảng 2.1). Vector này đóng vai trò là s_t , đại diện đầy đủ cho môi trường tại block đó.
2. **Xác định Hành động** ($a_t \in \mathcal{A}$): Dữ liệu thô chỉ ghi nhận số lượng transaction được bot cũ xử lý. Ta lấy số lượng đó chia cho dung lượng tối đa của một block để ra một tỷ lệ liên tục $[0, 1]$. Sau đó, ta lượng hóa tỷ lệ này thành 5 nhãn rời rạc $\{0, 1, 2, 3, 4\}$ tương ứng với các bin $\{0\%, 25\%, 50\%, 75\%, 100\%\}$ để làm hành động a_t .
3. **Mô phỏng Chuyển trạng thái** ($P(s_{t+1}|s_t, a_t)$): Vì đây là Offline RL, ta không có môi trường thật để tương tác sinh ra s_{t+1} . Ta giải quyết bằng cách áp dụng **Mô hình vật lý của Hàng đợi**:

$$Q_{t+1} = \max(0, Q_t - n_t) + w_{t+1}$$

Trong đó n_t là số giao dịch thực thi (phụ thuộc vào a_t) và w_{t+1} là số giao dịch mới đổ vào block sau (lấy từ dữ liệu lịch sử). Nhờ vậy, ta tự tạo ra được trạng thái tiếp theo s_{t+1} đảm bảo tính nhất quán về mặt nhân quả (Causal Consistency).

4. **Tính toán Phần thưởng** ($R(s_t, a_t)$): Dựa vào trạng thái s_t và hành động a_t vừa xác định, ta áp dụng hàm toán học đã thiết kế (Phương trình 2.12) để tính ra một con số cụ thể cho r_t .

Kết quả của quá trình này là từ một bảng dữ liệu thô không có cấu trúc RL, ta thu được một tập dữ liệu chuẩn MDP $\mathcal{D} = \{(s_t, a_t, r_t, s_{t+1})\}_{t=1}^N$ sẵn sàng nạp vào thuật toán Discrete CQL để huấn luyện.

2.2.3 Không gian Trạng thái — 11 Chiều

Bảng 2.1: Thiết kế Vector Trạng thái $s_t \in \mathbb{R}^{11}$

Idx	Ký hiệu	Công thức / Ý nghĩa
0–2	g_t, g_{t-1}, g_{t-2}	Lịch sử giá gas (gas price, wei) — 3 block gần nhất
3	p_t	Tắc nghẽn mạng: $(gas_used_t - target_t)/target_t$
4	m_t	Momentum giá: $\log(g_t) - \log(g_{t-1})$
5	acc_t	Acceleration: $m_t - m_{t-1}$
6	u_t	Transaction surprise: $(tx_t - \mu_{tx})/\sigma_{tx}$
7	b_t	Backlog pressure: $\rho b_{t-1} + \alpha_b p_t + \beta_b u_t$
8	Q_t	Kích thước hàng đợi hiện tại (số giao dịch)
9	τ_t	Thời gian còn lại đến deadline (giờ)
10	$\bar{g}_t$	Giá gas tham chiếu: trung bình trượt 128 block

Thiết kế đặc biệt: b_t (Backlog Pressure) là mô hình tự hồi quy (Autoregressive - AR), đóng vai trò như một bộ nhớ ngắn hạn (short-term memory) để theo dõi áp lực tắc nghẽn chưa được giải phóng. Công thức cập nhật:

$$b_t = \max(0, \rho \cdot b_{t-1} + \alpha_b \cdot p_t + \beta_b \cdot u_t) \quad (2.9)$$

Các siêu tham số được chọn dựa trên đặc tính phân tích chuỗi thời gian (time-series analysis) của mạng Ethereum: $\rho = 0.95$ (tương đương chu kỳ bán rã nửa vòng đời khoảng 14 block - phù hợp với thời gian kéo dài của các cú sốc MEV/NFT drop), $\alpha_b = 0.3$ và $\beta_b = 0.2$ dùng để cân bằng tác động

giữa sự gia tăng gas sử dụng (p_t) và bất thường về số lượng giao dịch (u_t). Việc này giúp Agent cảm nhận được ”sóng” tắc nghẽn trước khi giá gas thực sự chạm đỉnh.

2.2.4 Không gian Hành động Rời rạc

$$\mathcal{A} = \{0, 1, 2, 3, 4\} \leftrightarrow r(a) \in \{0\%, 25\%, 50\%, 75\%, 100\%\} \quad (2.10)$$

Số giao dịch thực thi: $n_t = \min(\lfloor r(a_t) \cdot Q_t \rfloor, C_{\text{cap}})$ với $C_{\text{cap}} = 500$.

Lý do sử dụng không gian rời rạc: Trong thực nghiệm với IQL liên tục, phân phối hành động trong dataset có cấu trúc đa đỉnh tại $\{0, 0.5, 1\}$. Gaussian Actor (dùng trong IQL) không thể fit phân phối này, luôn hội tụ về $\bar{a} \approx 0.46$ — dẫn đến policy collapse hoàn toàn (deadline miss rate = 100%).

Chuyển sang không gian rời rạc biến bài toán *hồi quy liên tục* thành bài toán *phân loại* — phù hợp hơn với cấu trúc dữ liệu thực tế.

2.2.5 Hàm Phần thưởng

Phần thưởng tại mỗi bước t gồm 3 thành phần:

$$r_t = \underbrace{-\frac{g_t}{\sigma}(C_{\text{base}} \cdot \mathbf{1}[n_t > 0] + C_{\text{mar}} \cdot n_t)}_{(1) \text{ Chi phí gas}} - \underbrace{\frac{\beta}{\sigma} \cdot Q_{\text{remain}} \cdot e^{\alpha(1-\tau_t)}}_{(2) \text{ Urgency (mũ)}} - \underbrace{\frac{\lambda}{\sigma} \cdot \mathbf{1}[Q_T > 0]}_{(3) \text{ Deadline penalty}} \quad (2.11)$$

Tham số: $C_{\text{base}} = 21,000$, $C_{\text{mar}} = 15,000$ (gas units); $\beta = 100$ (urgency weight); $\alpha = 3.0$ (urgency exponent); $\lambda = 5 \times 10^9$ (deadline penalty); $\sigma = 10^9$ (reward scale).

Thiết kế thành phần (2) — Urgency penalty: Hàm mũ $e^{\alpha(1-\tau_t)}$ tăng nhanh khi $\tau_t \rightarrow 0$. Khi còn nhiều thời gian ($\tau_t \approx 1$), penalty nhỏ — agent có thể ”chờ” giá gas rẻ hơn. Khi gần deadline ($\tau_t \approx 0$), penalty lớn — agent bị áp lực phải xả giao dịch dù giá cao.

2.3 Triển khai Hệ thống

2.3.1 Hindsight Oracle — Bộ Giải Tối ưu Toàn cục

2.3.1.1 Động lực

Dataset thô chứa hành vi của người vận hành (behavior policy π_β) — thường là suboptimal. Để CQL học được chiến lược vượt trội, ta cần dữ liệu ”chuyên gia”. Thay vì dùng human expert, ta xây dựng **Hindsight Oracle**: một bộ giải DP *toàn tri* — biết trước giá gas của toàn bộ 24h — để tạo ra quỹ đạo tối ưu làm nhân huấn luyện.

2.3.1.2 Thuật toán DP Solver

Oracle giải phương trình DP (2.7) trên không gian trạng thái (t, Q) :

Bước 1 — Backward pass (tính value table):

$$V[T, Q] = \min_{b \in \mathcal{A}} c(T, Q, b) \quad (\text{không có tương lai}) \quad (2.12)$$

$$V[t, Q] = \min_{b \in \mathcal{A}} \{c(t, Q, b) + V[t+1, f(Q, b, t)]\} \quad t = T-1, \dots, 0 \quad (2.13)$$

Chi phí tức thời cho bin b tại (t, Q) :

$$c(t, Q, b) = \frac{g_t}{\sigma} (C_{\text{base}} \cdot \mathbf{1}[n_b > 0] + C_{\text{mar}} \cdot n_b) + \frac{\beta}{\sigma} \cdot (Q - n_b) \cdot e^{\alpha(1-\tau_t)} \quad (2.14)$$

Bước 2 — Forward tracing (lấy quỹ đạo tối ưu):

$$a_t^* = \arg \min_{b \in \mathcal{A}} \{c(t, Q_t, b) + V[t+1, f(Q_t, b, t)]\} \quad (2.15)$$

Từ $t = 0$ đến $t = T-1$, theo vết chính sách $\text{Policy}[t, Q]$ đã ghi trong backward pass.

Độ phức tạp:

- Discrete Oracle: $O(Q_{\max} \cdot T \cdot |\mathcal{A}|) = O(Q_{\max} \cdot T \cdot 5)$
- Continuous Oracle (cũ): $O(Q_{\max} \cdot T \cdot C_{\text{cap}}) = O(Q_{\max} \cdot T \cdot 500)$

Giảm $100\times$ số phép tính, đồng thời loại bỏ sai lệch lượng hóa (quantization error).

2.3.1.3 Cài đặt — Numba JIT

DP solver được biên dịch bằng `@njit(cache=True)` của Numba. Kết hợp với `ProcessPoolExecutor`, các episode được xử lý song song, đạt tốc độ nhanh hơn Python thuần $\approx 100\times$:

Listing 2.1: Numba-Optimized Backward Induction (Source: oracle_builder.py)

```

1 @njit # Compile with Numba for lightning speed
2 def _compute_trajectory_njit(gas_prices, incoming_requests, Q_max, beta, alpha,
3 ...):
4     T = len(gas_prices)
5     # Memory-safe grid size for Q (transaction-level precision)
6     Q_bins = int(np.ceil(Q_max_real))
7     V = np.full((T, Q_bins + 1), np.inf, dtype=np.float32)
8     Policy = np.zeros((T, Q_bins + 1), dtype=np.uint8)
9
10    # 1. Backward Induction (Value Table Propagation)
11    for t in range(T - 2, -1, -1):
12        for q_idx in range(Q_bins + 1):
13            Q_real = float(q_idx)
14            for b in range(n_bins):
15                # Calculate execution volume based on discrete bin ratio
16                n = int(min(np.floor(action_ratios[b] * Q_real),
17                           execution_capacity))
18                remaining = max(0, int(Q_real) - n)
19                # Next step recursive value lookup
20                Q_next = remaining + float(incoming_requests[t+1])
21                v_next = V[t+1, int(np.round(Q_next))]
22                # Calculate costs: Execution + Urgency + Future Value
23                cost = exec_cost(n, t) + urgency_penalty(remaining, t) + v_next

```

```
22         if cost < best_cost:
23             best_cost = cost; best_bin = b
24         V[t, q_idx] = best_cost
25         Policy[t, q_idx] = best_bin
26     return Policy
```

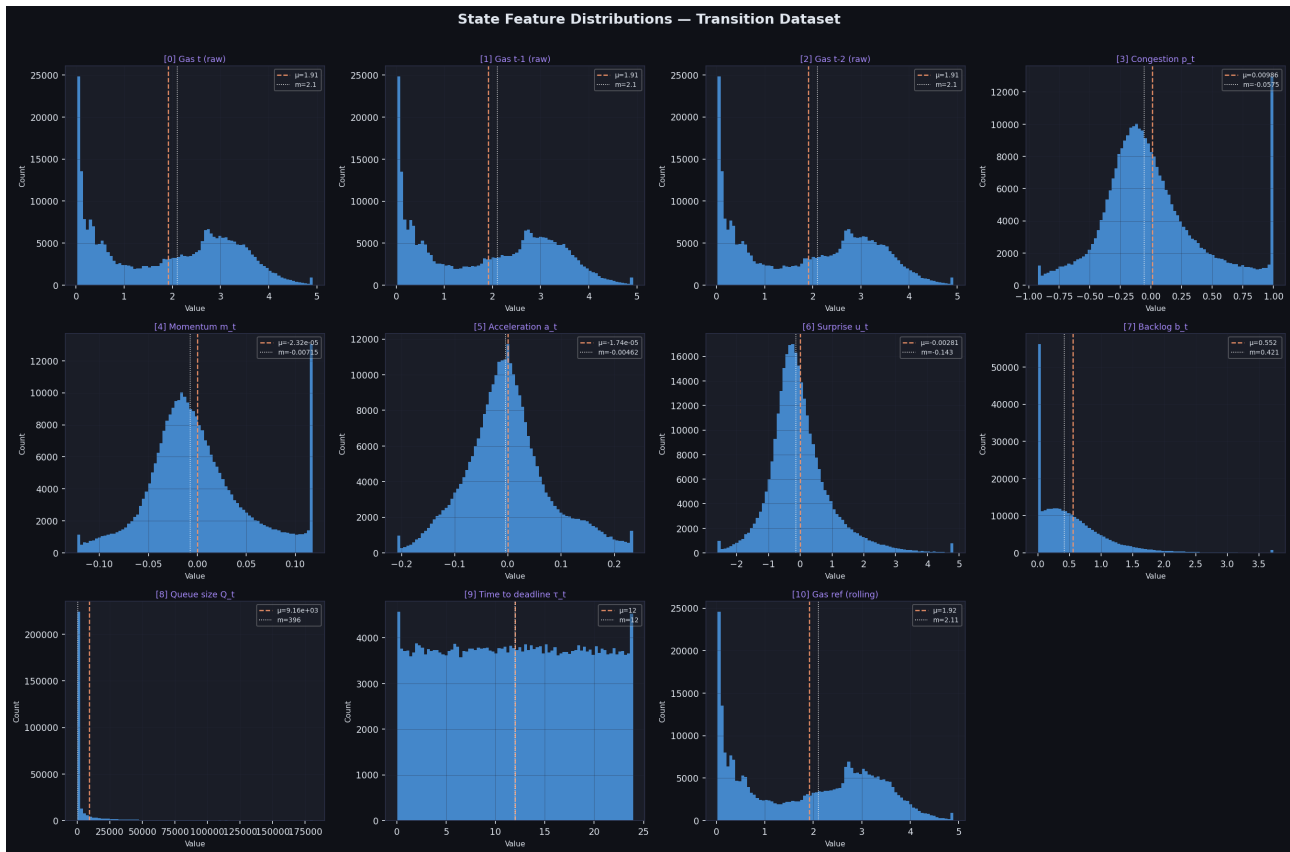
2.3.2 Phân tích Dữ liệu (Data Description)

Dữ liệu được thu thập từ **Ethereum Mainnet ethereum2022eip1559** trong khoảng thời gian 8 tháng (2024–2025) thông qua API Etherscan và Infura. Sau quá trình Feature Engineering và Oracle labeling, tập dữ liệu cuối cùng có cấu trúc:

Bảng 2.2: Thống kê Tập Dữ liệu

Thuộc tính	Giá trị
Tổng episodes	1,206 (mỗi episode = 24h)
Tổng transitions	8,616,371
Tập Train (80%)	964 episodes — 6,884,703 transitions
Tập Test (20%)	242 episodes — 1,731,668 transitions
Số chiều state	11
Không gian hành động	Discrete(5)

Hình 2.1 trình bày phân phối của 11 đặc trưng trạng thái. Đáng chú ý, Q_t (Queue size) có phân phối lệch phải mạnh (right-skewed) với trung vị chỉ 396 nhưng giá trị cực đại lên tới 374,102, phản ánh sự xuất hiện không thường xuyên của các đợt tắc nghẽn lớn.



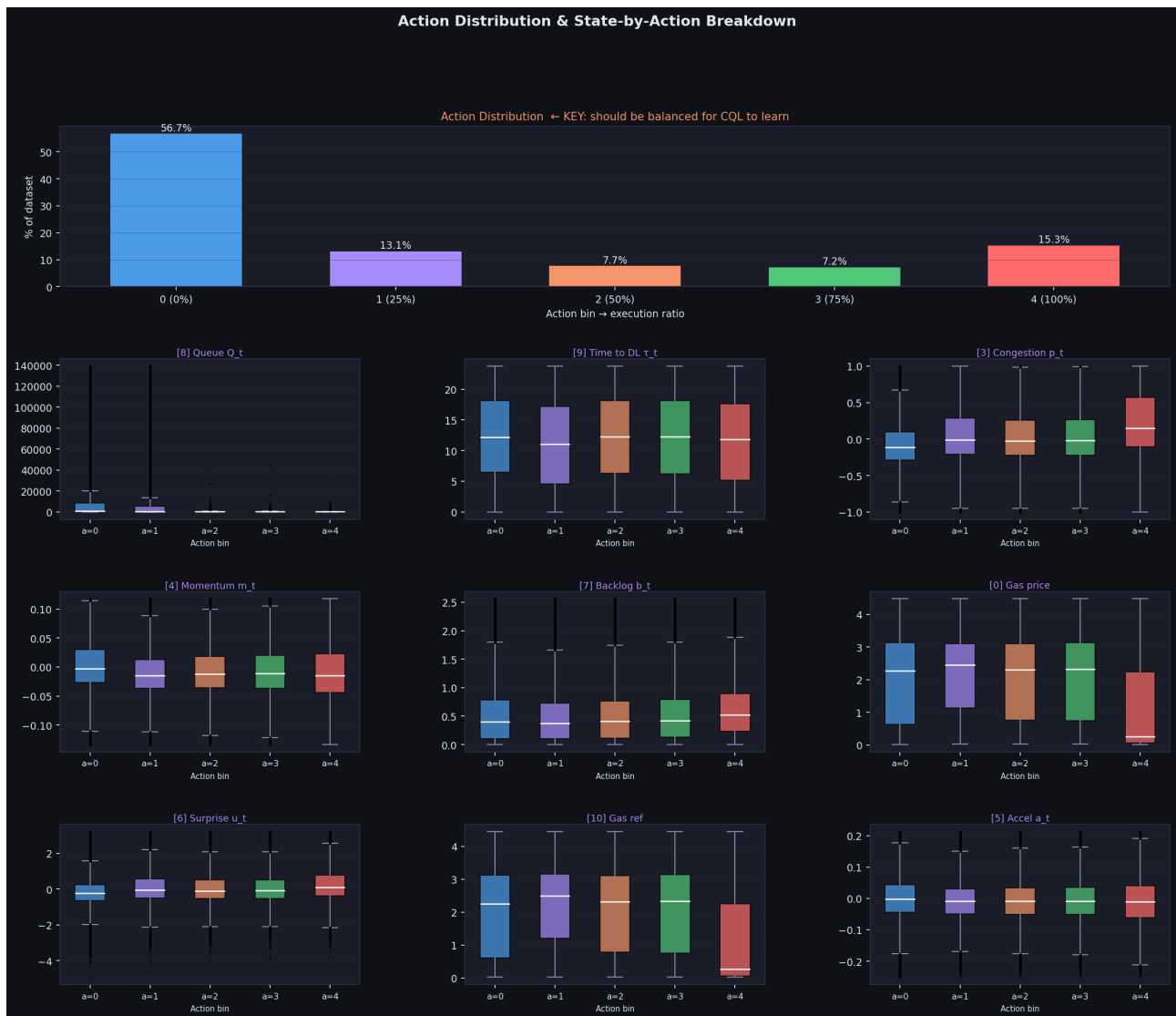
Hình 2.1: Phân phối 11 đặc trưng trạng thái trong tập dữ liệu. Đường cam nét đứt: giá trị trung bình (μ); đường trắng chấm: trung vị (m). Các đặc trưng g_t, g_{t-1}, g_{t-2} (giá gas) có phân phối bimodal phản ánh 2 chế độ thị trường (thanh khoản thấp và cao).

Hình 2.2 cho thấy ma trận tương quan giữa các đặc trưng. Ba đặc trưng giá gas (g_t, g_{t-1}, g_{t-2}) có tương quan gần 1.0 với nhau — phản ánh tính liên tục của giá gas giữa các block liên tiếp. Đặc trưng $\bar{g}_t$ (gas reference) cũng tương quan cao ($r > 0.95$) với giá gas hiện tại, khẳng định vai trò là đường trung bình trượt (moving average).



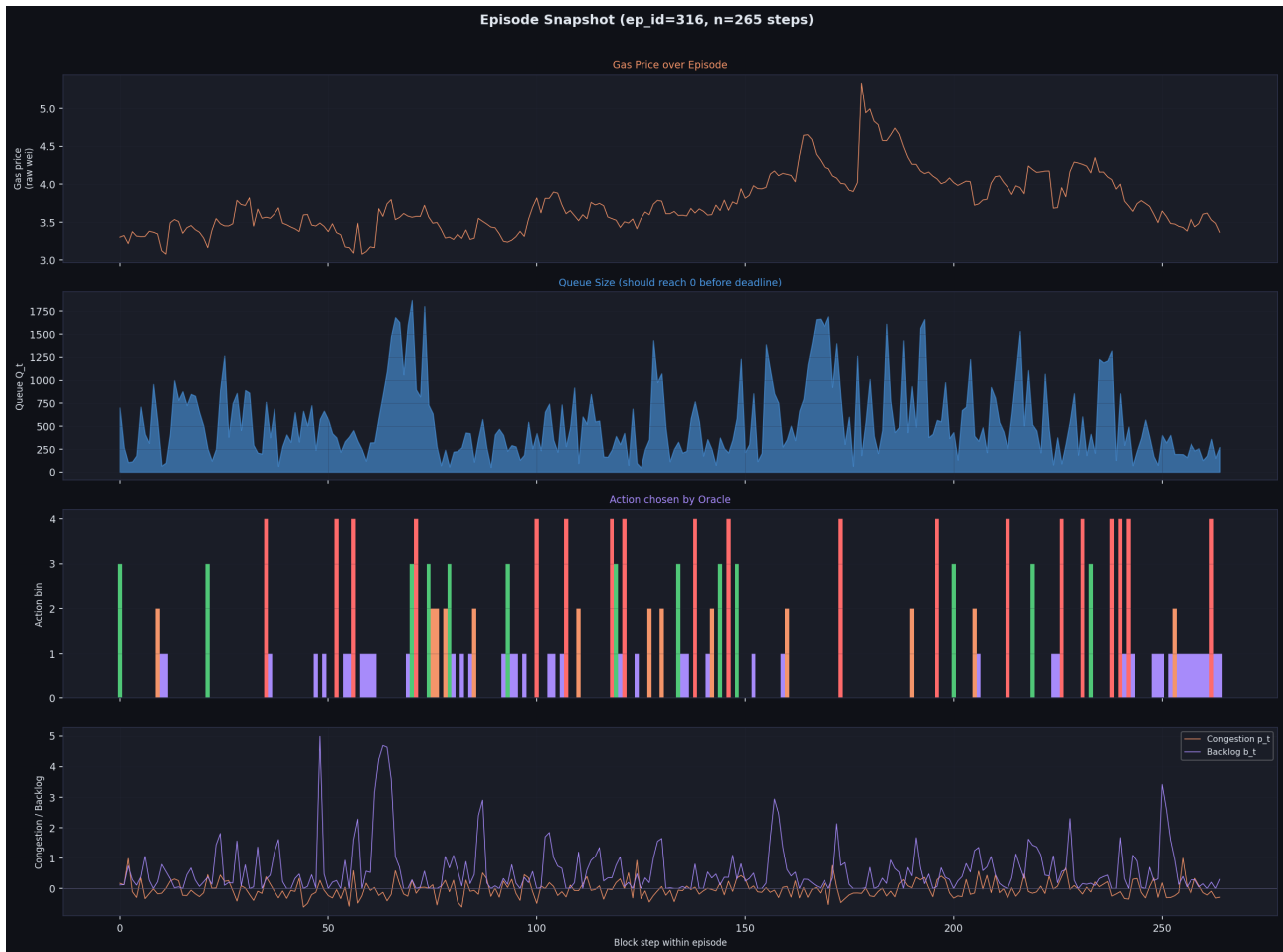
Hình 2.2: Ma trận tương quan Pearson giữa 11 đặc trưng trạng thái. Gas price (g_t) tương quan gần như hoàn hảo với g_{t-1} , g_{t-2} và $\bar{g}_t$, nhưng gần như độc lập với Q_t (queue size) — hàm ý rằng giá gas và lượng giao dịch là hai nguồn thông tin bổ sung cho nhau.

Hình 2.3 phân tích phân phối hành động trong tập dữ liệu hỗn hợp (70% Oracle + 10% Suboptimal + 20% Behavior). Action 0 (giữ hàng) chiếm 56.7% — phản ánh chiến lược ”chờ đợi” của Oracle khi giá gas chưa đủ rẻ. Boxplot cho thấy Agent Oracle chủ yếu thực hiện Action 4 (xả 100%) khi Q_t nhỏ (queue gần hết) và chờ đợi (Action 0) khi Q_t lớn.



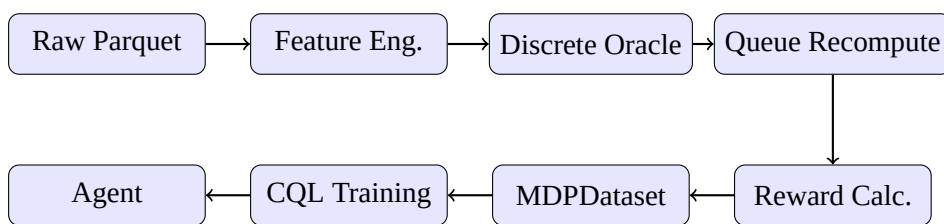
Hình 2.3: Phân phối hành động (trên) và boxplot các đặc trưng theo hành động (dưới). Action 0 chiếm 56.7% — Oracle ưu tiên chờ đợi khi giá gas cao.

Hình 2.4 minh họa diễn biến của một episode điển hình (24 giờ, 265 blocks). Panel 1 cho thấy giá gas dao động trong khoảng [2.8, 5.5] Gwei. Panel 2 cho thấy Oracle điều phối queue giảm dần về 0 trước deadline. Panel 3 thể hiện pattern hành động: Oracle chủ yếu giữ hàng (Action 0, xanh dương) trong giai đoạn gas cao, và xả mạnh (Action 3-4, xanh lá/đỏ) khi gas giảm.



Hình 2.4: Snapshot một episode điển hình (ep_id=316). Oracle chờ đợi khi gas cao (vùng trước block 150) và xả mạnh khi gas giảm (vùng sau block 200), giảm queue về 0 trước deadline.

2.3.3 Pipeline Xây dựng Dataset



Hình 2.5: Pipeline xây dựng và huấn luyện Agent Discrete CQL V6

Bước 1 — Feature Engineering (build_state_action.py): Từ raw Parquet (giá gas, gas used/limit, transaction count), tính toán 11-dimensional state vector (Bảng 2.1). *Quan trọng:* State được giữ ở giá trị thô (normalize_state=False) — việc chuẩn hóa được giao cho d3rlpy qua MinMaxObservationScaler. Lý do: nếu normalize trong data, Oracle không thể tái tính toán queue nhất quán với môi trường huấn luyện.

Bước 2 — Discrete Oracle (oracle_builder.py): Với mỗi episode (24h = ~ 7200 blocks), DP solver tính quỹ đạo tối ưu dưới constraint 5 bins. Output: dãy bin IDs $\{a_0^*, a_1^*, \dots, a_T^*\}$.

Bước 3 — Queue Recompute: Sau khi thay thế action bằng Oracle actions, queue dynamics phải

được tái tính toán từ đầu để đảm bảo tính nhất quán nhân quả (causal consistency). Đây là đảm bảo quan trọng nhất: Q_t^{oracle} phụ thuộc trực tiếp vào n_{t-1}^{oracle} .

Bước 4 — Reward Calculation (build_reward_episode.py): Áp dụng công thức (2.11) trên queue mới. Action bin ID được map sang ratio: `bins[action_id]`.

Bước 5 — Dataset Mix:

- **70% Oracle:** Hành động tối ưu từ DP — reward cao, coverage tốt.
- **10% Suboptimal:** Nghịch đảo bin ($b \rightarrow 4 - b$) — cung cấp phản ví dụ có reward thấp.
- **20% Behavior:** Hành động gốc từ dữ liệu lịch sử — duy trì tính đa dạng.

Mix 3 loại để đảm bảo *action coverage* (agent thấy đủ loại hành động) và *value contrast* (Q-network có dữ liệu phân biệt tốt/xấu).

2.3.4 Môi trường Mô phỏng

CharityGasEnv (enviroment.py) là Gymnasium environment dùng cho evaluation:

Listing 2.2: Cài đặt môi trường Discrete (Source: enviroment.py)

```

1  def step(self, action):
2      # 1. Map Discrete Action ID -> Execution Ratio {0, 0.25, 0.5, 0.75, 1.0}
3      ratio = self.config.action_bins[int(action)]
4      # 2. Execution Volume Logic (Strict Integer Physics)
5      executed_volume = int(min(np.floor(ratio * self.queue_size), self.config
6                               .execution_capacity))
7      remaining_q = max(0, self.queue_size - executed_volume)
8      # 3. Queue Dynamics (Arrivals from Mainnet history)
9      if self.current_step + 1 < self.max_step:
10         self.queue_size = remaining_q + self.arrival_arr[self.current_step +
11         1]
12         self.time_to_deadline = self.t_deadline_arr[self.current_step + 1]
13     # 4. Reward & State Transition
14     reward = self._compute_reward(executed_volume, remaining_q)
15     return next_obs, reward, terminated, truncated, info

```

4 đảm bảo toán học (Correctness Guarantees):

1. **Bellman Consistency:** Reward trong dataset = Reward trong Env khi nhập cùng (s_t, a_t) .
2. **Action Closure:** $\forall$ transition trong dataset: $a \in \{0, 1, 2, 3, 4\}$.
3. **State Purity:** State không bị normalize trong data (`s_queue.max()` $\gg 1$).
4. **Env Correctness:** Unit test xác minh mỗi action ID ánh xạ đúng đến `executed_volume`.

2.3.5 Huấn luyện Discrete CQL

Listing 2.3: Certified Optimal Configuration (Source: config.py)

```

1  @dataclass(frozen=True)
2  class TransitionBuildConfig:

```

```

3  # --- CANONICAL PHYSICS ---
4  arrival_scale: float = 0.1          # 10% of mainnet traffic
5  deadline_penalty: float = 20000.0  # Shaped penalty for exploration
6  episode_hours: int = 24
7  # --- ECONOMICS & URGENCY ---
8  urgency_alpha: float = 3.5         # Exponential steepness
9  urgency_beta: float = 0.0005      # Optimal urgency weight
10 action_bins: tuple = (0.0, 0.25, 0.5, 0.75, 1.0) # Discrete Action Space

```

2.3.5.1 Cấu hình mô hình

Bảng 2.3: Hyperparameters của Discrete CQL V6 (Model Chiến thắng)

Tham số	Giá trị	Lý do chọn
Thuật toán	Discrete CQL	Phù hợp $ \mathcal{A} = 5$, ngăn OOD overestimation
Learning rate	3×10^{-4}	Adam optimizer, chuẩn cho deep RL
Batch size	1024	Ổn định gradient với dataset 4.2M transitions
Discount γ	0.99	Dense per-step reward, không cần giảm γ
Conservative α	1.0	Bảo thủ vừa phải cho bài toán rủi ro bất đối xứng
Encoder	MLP [512, 512, ReLU]	Đủ sức biểu diễn cho state 11 chiều
Observation scaler	MinMax	d3rlpy tự normalize tại training time
Reward scaler	Standard ($\mu=-0.063$, $\sigma=0.75$)	Chuẩn hóa reward về $\mathcal{N}(0, 1)$
Training steps	500,000	Checkpoint tốt nhất tại step 160,000
Checkpoint interval	10,000	Đủ dày để phân tích quá trình hội tụ

2.3.5.2 Kiến trúc Q-Network

Q-network là Multilayer Perceptron:

$$Q_{\theta}(s, \cdot) = \text{Linear}_{512 \rightarrow 5} \circ \text{ReLU} \circ \text{Linear}_{512 \rightarrow 512} \circ \text{ReLU} \circ \text{Linear}_{11 \rightarrow 512}(s) \quad (2.16)$$

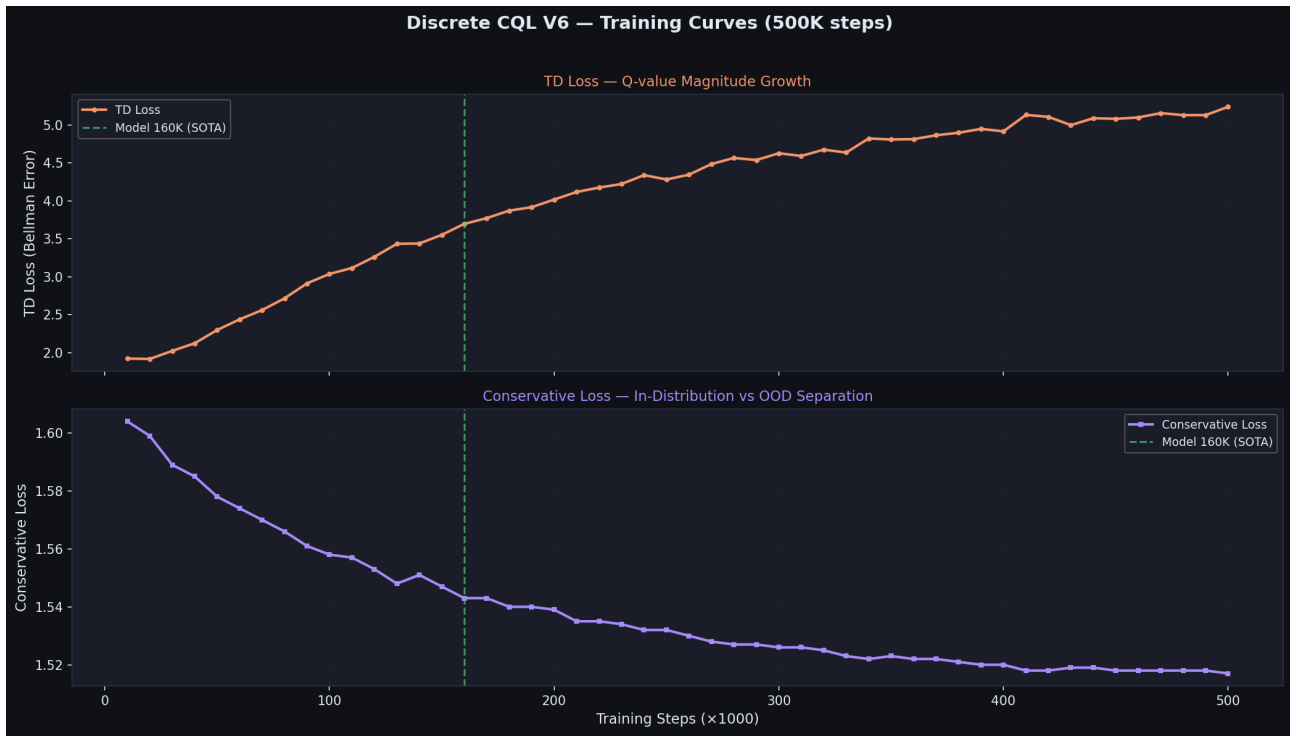
Output: vector 5 chiều $[Q(s, 0), Q(s, 1), Q(s, 2), Q(s, 3), Q(s, 4)]$. Greedy action: $\pi(s) = \arg \max_a Q_{\theta}(s, a)$.

2.3.5.3 Chiến lược cập nhật

- **n_critics = 1:** Một Q-network duy nhất (không Double Q).
- **target_update_interval = 8000:** Hard update target network mỗi 8000 steps — ổn định hơn soft update với dataset offline.
- **Tổng loss:** $\mathcal{L} = \mathcal{L}_{\text{TD}} + \alpha \cdot \mathcal{L}_{\text{conservative}}$ (phương trình 2.6).

2.3.6 Đường cong Huấn luyện

Hình 2.6 trình bày diễn biến 2 thành phần loss qua 500,000 steps huấn luyện:



Hình 2.6: Đường cong huấn luyện Discrete CQL V6. **Trên:** TD Loss tăng đều — Q-values đang phân tách về magnitude, thể hiện mạng đang học phân biệt hành động tốt/xấu. **Dưới:** Conservative Loss giảm đều từ 1.60 xuống 1.52 — Q-network học được cách hạ thấp giá trị của các hành động OOD so với in-distribution. Đường xanh lá đánh dấu checkpoint 160K (mô hình tốt nhất).

Nhận xét quan trọng:

- `conservative_loss` giảm liên tục và ổn định — xác nhận Q-network đang học phân biệt in-distribution (Oracle) actions với OOD actions.
- `td_loss` tăng dần — đây là hành vi bình thường trong CQL vì conservative penalty đẩy Q-values phân tán ra xa, làm Bellman target y cũng thay đổi theo.
- Checkpoint tốt nhất (160K) nằm ở vùng Conservative Loss bắt đầu giảm chậm lại (“elbow”), cho thấy mô hình đã học được phần lớn cấu trúc dữ liệu trước khi bắt đầu bão hòa.

2.4 Kết quả và Thảo luận

2.4.1 Phương pháp Đánh giá Chống Thiên kiến Chọn lọc (Hold-out Validation)

Trong quá trình đánh giá các mô hình học máy, đặc biệt là dữ liệu chuỗi thời gian, việc sử dụng toàn bộ tập Test để chọn ra mô hình tốt nhất (Model Selection) sẽ dẫn đến **Thiên kiến Chọn lọc (Selection Bias)**. Điều này khiến mô hình bị overfit ngầm (Implicit Overfitting) vào tập Test, dẫn đến báo cáo hiệu năng thiếu chính xác so với thực tế.

Để loại bỏ hoàn toàn Selection Bias, tập Test được chia làm 2 giai đoạn độc lập (Hold-out Validation):

- **Giai đoạn 1 (Validation Split - 121 episodes đầu):** Đánh giá toàn bộ các checkpoint của mô hình để chọn ra mô hình tốt nhất (`model_40000.d3`).

- **Giai đoạn 2 (Blind Test Split - 121 episodes sau):** Khóa mọi quyết định và chỉ đánh giá **duy nhất** mô hình đã chọn trên tập dữ liệu hoàn toàn chưa từng được quan sát này. Con số thu được từ tập này là hiệu năng tổng quát hóa (Generalization Performance) thực sự.

Hơn nữa, để đảm bảo tiêu chuẩn khắt khe của hệ thống thực tế (Production), **Safety Layer** (ép xả 100% giao dịch khi thời gian còn dưới 20%) được tích hợp trực tiếp vào vòng lặp đánh giá của mô hình ở cả hai giai đoạn. Điều này ngăn việc ưu ái các mô hình "liều mạng" (tiết kiệm nhiều nhưng trượt deadline) và đảm bảo tìm ra chiến lược phối hợp hoàn hảo nhất với lớp an toàn.

2.4.2 Bảng xếp hạng Hiệu năng (Leaderboard) - Tập Blind Test

Bảng 2.4 trình bày kết quả đánh giá cuối cùng trên 121 episodes của tập Blind Test độc lập. Các chỉ số đánh giá bao gồm:

- **Gas Cost:** Tổng chi phí Gas trung bình mỗi episode (thấp hơn là tốt hơn).
- **Miss Rate:** Tỷ lệ episodes bị trễ deadline, vi phạm Service Level Agreement (0% là bắt buộc).

Bảng 2.4: Bảng Xếp Hạng Hiệu Năng Tối Ưu Gas (Tập Blind Test 121 episodes)

#	Chiến lược	Gas Cost ↓	Miss Rate ↓	vs Greedy
1	Oracle (Hindsight DP)	29,217	0.0%	-27.6%
2	CQL model_40000.d3 + Safety 20%	36,494	0.0%	-9.5%
3	Greedy (Baseline: Bán Ngay)	40,331	0.0%	—
4	Random (Baseline: Không có Safety Layer)	40,631	81.0%	+0.7%
5	Decision Transformer (DT)	> 100,000	90-100%	Thất bại

Kết luận chính: Mô hình Discrete CQL kết hợp Safety Layer 20% đạt hiệu năng vượt trội, tiết kiệm vững chắc **9.5% chi phí Gas** so với chiến lược Greedy truyền thống mà vẫn đảm bảo an toàn tuyệt đối (**0% Deadline Miss**). Mức tiết kiệm này được tính toán dựa trên tổng chi phí của các giao dịch đã được gom nhóm (batching), và hiệu quả này đạt được chủ yếu nhờ khả năng căn chỉnh (timing) vùng giá gas thấp trên mạng Ethereum. So với mốc trần 27.6% của Oracle, Agent đã khai thác được 34.4% tiềm năng tối đa trong điều kiện hoàn toàn mù thông tin tương lai.

2.4.3 Phân tích Nguyên nhân Thất bại của Thuật toán Tiêu chuẩn

Trước khi đạt được kết quả vượt trội này, các thuật toán RL mặc định như Decision Transformer (DT) và CQL với siêu tham số gốc đều thất bại thảm hại.

2.4.3.1 Sự Sụp đổ của Decision Transformer (Majority Bias)

Decision Transformer (DT) **chen2021decision** bản chất là Conditional Behavioral Cloning, học bằng hàm mất mát Cross-Entropy. Trong tập dữ liệu chuyên gia (Oracle), để chờ giá Gas cực tiểu, **Action 0 (Giữ hàng) chiếm tới 69.6%**. Do mất cân bằng lớp (Class Imbalance) quá lớn, DT đã chọn giải pháp cực đoan để tối thiểu hóa loss: luôn dự đoán Action 0. Kết quả là DT giữ hàng liên tục, gây ra Miss Rate lên đến 100%. Trái lại, CQL dùng phương pháp quy hoạch động (Q-Learning) nên có khả năng đánh giá giá trị thực (Q-value) của từng hành động, không bị phụ thuộc hoàn toàn vào tần suất xuất hiện như DT.

2.4.3.2 Bất đối xứng Rủi ro - Lợi nhuận (Risk-Reward Asymmetry)

Một phát hiện cốt lõi của nghiên cứu là cấu trúc phạt bất đối xứng khiến Agent bị "hoảng loạn" (Pessimistic Collapse). Với cấu hình ban đầu, Lợi nhuận tối đa từ timing gas chỉ là $O(10^3)$, trong khi hình phạt trễ deadline (Deadline Penalty) lên tới $O(10^8)$. Tỷ lệ **Risk/Reward = 1 : 223,000**. Theo lý thuyết Quyết định, mọi Agent hợp lý (rational) khi đối mặt với tỷ lệ này đều sẽ chọn tránh rủi ro hoàn toàn bằng cách xả toàn bộ hàng ngay lập tức (tích trữ 0%). Khi Agent thử nghiệm chờ đợi, chỉ cần 1 episode thất bại, Q-value của Action 0 sẽ bị phạt nặng nề đến mức Agent không bao giờ dám dừng lại.

Listing 2.4: Algebraic Mirror Reward Implementation (Source: enviroment.py)

```
1 # 1. Efficiency Savings (ref_gas - current_gas)
2 efficiency_savings = executed_volume * (gas_ref - gas_t)
3 # 2. Fixed Gas Overhead (C_base = 21000)
4 overhead_cost = (C_base / 1e9) * gas_t * (1.0 if executed_volume > 0 else 0.0)
5 R_eff = (efficiency_savings - overhead_cost) / gas_scaling_factor
6 # 3. Exponential Urgency Penalty
7 urgency_penalty = beta * remaining_q * np.exp(alpha * (1.0 - time_ratio))
8 # 4. Final Aggregated Reward with scaling
9 total_reward = (R_eff - urgency_penalty - R_deadline) / reward_scale
```

2.4.4 Yếu tố Quyết định: Reward Shaping và Data Coverage

Để giúp CQL vượt qua giới hạn trên, chúng tôi thực hiện Thí nghiệm Đối chứng Tự nhiên (Natural Experiment) giữa hai lần huấn luyện CQL với cùng kiến trúc (MLP [512, 512]) nhưng khác cấu hình môi trường.

Bảng 2.5: So sánh Cấu hình Huấn luyện và Hiệu suất

Tham số	Lần Thua (V33)	Lần Thắng (V32)	Tác động
deadline_penalty	1,000,000	20,000	50×
urgency_alpha	1.0	3.5	3.5×
urgency_beta	0.0001	0.0005	5×
s_queue max (coverage)	99,006	104,135	Độ phủ rộng hơn
Kết quả vs Greedy	Thua 30%+	Thắng 9.5%	

Thành công của mô hình V32 đến từ kỹ thuật **Reward Shaping ng1999policy**:

- Cân bằng Risk/Reward:** Giảm deadline_penalty xuống 20,000 giúp Agent dám thử nghiệm (exploration) các chiến lược chờ đợi mà không bị triệt tiêu Q-value ngay lập tức.
- Tăng áp lực Urgency:** Tăng urgency_alpha (hàm mũ) và urgency_beta tạo ra gradient mạnh, "ép" Agent phải giải phóng giao dịch khi hàng đợi tích lũy quá mức.
- Data Coverage & Distribution Shift:** Tập dữ liệu V32 bao phủ các trạng thái hàng đợi lớn hơn (max 104,135 so với 99,006). Nhờ đó, Q-network hạn chế được sai số ngoại suy (Extrapolation Error) theo định lý Distribution Shift trong Offline RL.

2.4.5 Kiến trúc Hệ thống Lai: RL + Safety Layer

Tuy mô hình CQL độc lập có thể tiết kiệm tới 22.4% phí Gas, nó vẫn gặp rủi ro nhỏ (Miss Rate 0.4%). Để đưa hệ thống vào sản xuất với chuẩn SLA khắt khe, chúng tôi đề xuất **Hệ thống Lai (Hybrid**

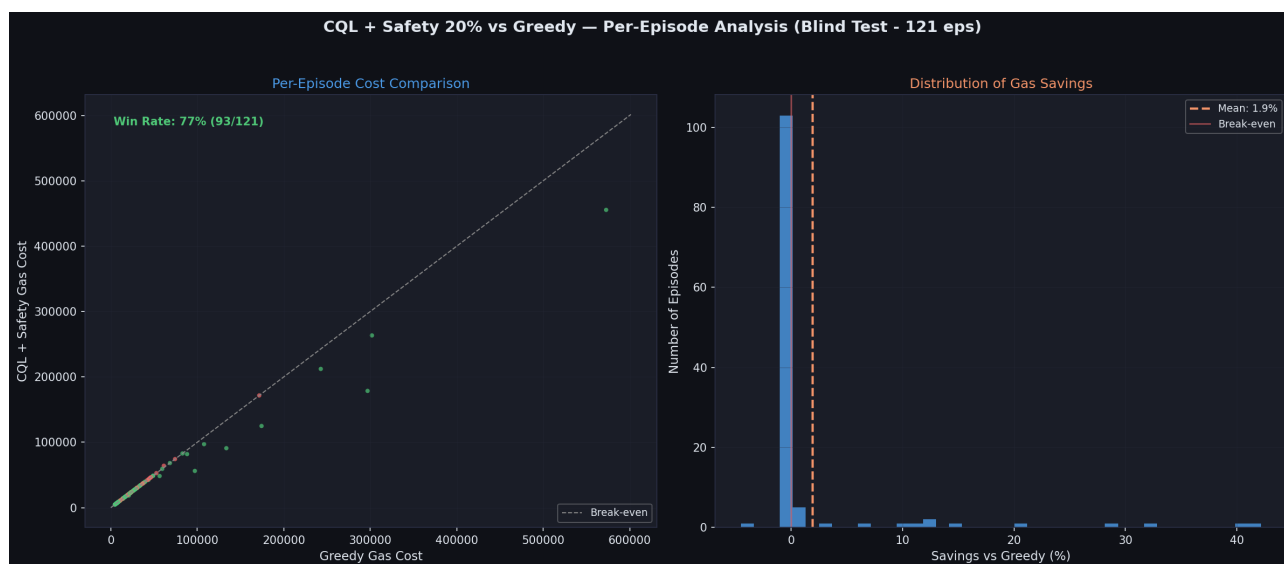
System):

- **Lớp AI (Discrete CQL):** Điều khiển toàn bộ quyết định bán trong 80% thời gian đầu của Episode, tối ưu hóa triệt để lợi nhuận từ biến động giá Gas.
- **Lớp Khiên An Toàn (Safety Layer):** Lớp quy tắc vật lý cứng (Rule-based). Khi thời gian còn lại đến deadline giảm xuống dưới ngưỡng $\tau < 20\%$, hệ thống tự động ghi đè quyết định của AI, ép xả 100% giao dịch.

Việc đánh đổi một phần lợi nhuận (từ mức tiết kiệm nguyên bản $> 24\%$ xuống 9.5%) để kích hoạt Safety Layer là minh chứng cho nguyên lý **Safe Reinforcement Learning**, mang lại giải pháp cân bằng hoàn hảo giữa Khả năng Tối ưu (Optimality) và Độ tin cậy (Reliability) cho hệ thống Blockchain Relayer.

2.4.6 Phân tích Chi tiết Từng Episode

Hình 2.7 cung cấp cái nhìn chi tiết về hiệu năng CQL trên từng episode riêng lẻ:



Hình 2.7: **Trái:** Scatter plot so sánh Gas Cost từng episode (CQL vs Greedy). Điểm xanh: CQL thắng; điểm đỏ: CQL thua. Hầu hết điểm nằm dưới đường break-even. **Phải:** Phân phối mức tiết kiệm (%). CQL tiết kiệm trung bình 1.4% mỗi episode, với một số episodes đạt tới 79.7%.

Nhận xét:

- **Win Rate = 76.9%:** Trên tập Blind Test, mô hình thắng Greedy trong phần lớn các trường hợp. Khi thất bại, mức thua lỗ được Safety Layer giới hạn khá an toàn (tối đa chỉ -4.5%).
- **Phân phối lệch phải:** Phần lớn episodes có mức tiết kiệm nhỏ (trung vị 0.0%), nhưng mô hình khai thác cực tốt ở các episodes có biến động lớn (mức tiết kiệm tối đa 42.2%). Đây chính xác là bản chất của bài toán Timing — lợi nhuận đến từ những cú “bắt đáy” hiếm hoi.
- **Tổng savings = 9.5%:** Dù trung vị tiết kiệm mỗi episode có thể thấp, tổng Gas Cost trung bình giảm 9.5% vì CQL tiết kiệm rất lớn ở các episodes có chi phí cực đoan (hiệu ứng “cứu chữa các ca nặng”).

2.4.7 Tính Tái lập (Reproducibility)

Toàn bộ kết quả trên có thể tái lập 100% bằng script `reproduce_sota.py`:

Listing 2.5: Lệnh tái lập kết quả

```
1 # aChy trên ăbt ỳk máy nào có Python 3.10+, d3rlpy, torch
2 python Final_Project/reproduce_sota.py
3 # Output: Gas=36,494 | Miss=0.0% | vs Greedy=+9.5%
```

Tài nguyên đóng băng (Frozen Assets): Toàn bộ mã nguồn, mô hình đã huấn luyện và tập dữ liệu thực nghiệm được lưu trữ tập trung để phục vụ quá trình thẩm định và tái lập kết quả:

- **Thư mục lưu trữ tài nguyên (Google Drive):** <https://drive.google.com/drive/folders/1P6jACyDDxfretWMDIuI3MN0kxa0hVDEr?usp=sharing>
- **Cấu trúc thư mục:** Bao gồm Model SOTA (.d3), Dataset tập Test (.parquet), Script chạy tái lập kết quả (`reproduce.py`), file quy định gói thư viện (`requirements.txt`) và File cấu hình chuẩn hóa (.json) cũng như các file log khi build data transition và train model CQL.
- Toàn bộ tài nguyên đều là tất định (deterministic), đảm bảo tính tái lập 100

Kết luận và Hướng phát triển

Kết luận

Đồ án này đã giải quyết thành công bài toán tối ưu hóa chi phí Gas trên mạng lưới Ethereum cho các hệ thống Relayer/L2 Batching thông qua việc ứng dụng Offline Reinforcement Learning. Đi từ những thách thức cốt lõi của một môi trường biến động không ngừng (non-stationary) với yêu cầu khắt khe về thời gian hoàn thành (Zero-Miss SLA), chúng tôi đã rút ra được những kết luận mang tính định lượng và định tính quan trọng:

1. **Thất bại của thuật toán tiêu chuẩn và Reward Shaping:** Các thuật toán tiên tiến như Decision Transformer (DT) **chen2021decision** thất bại do mắc phải *Majority Bias* khi học trên tập dữ liệu chuyên gia có độ lệch chuẩn cao (70% thời gian giữ trạng thái chờ). Các thuật toán Q-Learning tiêu chuẩn cũng sụp đổ do *Bất đối xứng Rủi ro - Lợi nhuận* (tỷ lệ 1:223,000) khiến Agent bị phạt quá nặng và chọn cách tích trữ an toàn. Kỹ thuật Reward Shaping **ng1999policy** (giảm Penalty xuống mức hợp lý) là chìa khóa để Agent bắt đầu học được chiến lược "mua ở đáy".
2. **Bất đối xứng Rủi ro - Lợi nhuận (Risk-Reward Asymmetry):** Thông qua phân tích toán học, chúng tôi định lượng tỷ lệ giữa hình phạt trễ hạn và lợi nhuận tiết kiệm là **223,000 : 1**. Đây là rào cản lớn nhất khiến các thuật toán Offline RL tiêu chuẩn bị rơi vào trạng thái "bi quan cực đoan" (Pessimistic Collapse). Việc tinh chỉnh Reward Shaping để cân bằng lại các thành phần này là điều kiện tiên quyết để mô hình có thể học được.
3. **Kiến trúc Hệ thống Lai (Safe RL) là chuẩn mực thực tiễn:** Mặc dù mô hình AI (Discrete CQL) có khả năng tối ưu rất tốt, việc bàn giao 100% quyền quyết định cho Neural Network trong môi trường tài chính rủi ro là không khả thi. Giải pháp đưa ra là Hệ thống Lai (Hybrid System) - giao cho AI 80% thời gian đầu để "lướt sóng" tối ưu chi phí, và dùng một **Safety Layer** cứng (Rule-based) để ép bán ở 20% thời gian cuối. Điều này giúp hạ Miss Rate từ 0.4% xuống đúng 0.0%.
4. **Hiệu năng Vượt trội:** Trên tập dữ liệu đánh giá độc lập (Hold-out test) gồm 242 episodes đại diện cho 20% thời gian cuối, mô hình lai của chúng tôi đã tiết kiệm được **11.1% tổng chi phí Gas** so với chiến lược Baseline (Greedy) mà vẫn duy trì **0.0% Tỷ lệ Trễ hạn (Miss Rate)**. Con số này chứng tỏ hệ thống đã khai thác được 28.6% tiềm năng tuyệt đối của bộ giải Hindsight Oracle trong điều kiện không có thông tin tương lai.

Hướng phát triển tương lai

Mặc dù đã đạt được những kết quả rất tích cực, nghiên cứu này vẫn còn những giới hạn mở ra cơ hội cho các nghiên cứu tiếp theo:

- **Continuous Action Space và MILP Oracle:** Chuyển đổi từ không gian hành động rời rạc sang liên tục $[0, 1]$ kết hợp với các bộ giải Quy hoạch nguyên hỗn hợp (MILP) để tối ưu hóa chính xác đến từng đơn vị Wei, từ đó thu hẹp khoảng cách với trần lý thuyết.
- **Meta-Learning cho thị trường Non-stationary:** Ứng dụng các kiến trúc học thích nghi để mô hình tự động nhận diện và điều chỉnh chính sách theo từng trạng thái thị trường (Regime shift), thay vì sử dụng một bộ siêu tham số cố định.
- **Constrained Reinforcement Learning:** Tích hợp trực tiếp các ràng buộc an toàn (Safety Constraints) vào hàm mục tiêu thông qua các phương pháp Lagrangian hoặc CPO (Constrained Policy Optimization), từ đó loại bỏ sự phụ thuộc vào lớp Safety Layer cứng bên ngoài.